

NOI 算法梳理

正文整理版

tzc_wk

原文发布：2022-08-08；更新：2023-05-03

目录

1. 字符串方向	5
1.1. KMP	5
1.2. Z 算法	6
1.3. AC 自动机	6
1.4. SA	6
1.5. Manacher	7
1.6. SAM	7
1.6.1. SAM 与后缀树	7
1.6.2. SAM 的建立	8
1.6.3. 一些后缀自动机的基本技巧	9
2. 图论方向（考察概率 100%）	9
2.1. 最短路	9
2.1.1. Bellman ford	9
2.1.2. Dijkstra	9
2.1.3. SPFA	10
2.1.4. Floyd	10
2.2. 最小生成树	10
2.2.1. Prim	10
2.2.2. Kruskal	10
2.2.3. Boruvka	10
2.3. 差分约束	10
2.4. 2-SAT	11
2.5. 同余最短路	11

2.6. 欧拉回路	11
2.7. 无/有向图连通性	12
2.7.1 Tarjan	12
2.7.2 VBCC/EBCC	13
2.7.3 圆方树	13
2.8. 数据结构优化建图	13
2.8.1. 线段树优化建图	14
2.8.2. 前后缀优化建图	14
2.8.3. 堆优化存边	14
3. 树论方向	14
3.1. LCA	14
3.2. 树链剖分	14
3.3. Kruskal 重构树	15
3.4. 虚树	15
3.5. 树上启发式算法	16
3.5.0 简介	16
3.5.1. dsu on tree (子树信息大小与 siz 同阶)	16
3.5.2. 长链剖分 (子树信息大小与 mxdep 同阶)	16
3.5.3 线段树合并 (维护的信息可通过线段树的合并操作得到)	17
3.5.4. set 启发式合并	17
3.6. 点分治与点分树	17
3.6.1. 点分治	17
3.6.2. 点分树	18
4. 数学方向 (考察概率 70%)	18
4.1 组合数与组合恒等式	18
4.2. 容斥原理	19
4.2.1. 二项式反演	20
4.2.2. Min-Max 反演	20
4.2.3. 卡特兰数与折线翻折模型	21
4.3. 单位根反演	21
4.4. 形式幂级数	22
4.4.1. 多项式乘法	22

4.4.2. 多项式基础操作	23
4.4.3. 生成函数	25
4.4.4. 拉格朗日插值	27
4.5. 集合幂级数	27
4.5.1. FWT	27
4.5.2. k 进制 FWT	29
4.5.3. 子集卷积	29
4.6. 线性代数	30
4.6.1. 高斯消元	30
4.6.2. 线性基	31
4.6.3. 行列式	31
4.6.4. 矩阵求逆	32
4.6.5. 矩阵树定理	32
4.6.6. BEST 定理	33
4.6.7. LGV 引理	33
4.7. 群论	33
5. 数据结构方向	34
5.1. 五种基础数据结构	34
5.2. 线段树的高级用法	35
5.2.1. 动态开点线段树与标记永久化	35
5.2.2. 李超线段树	35
5.2.3. 势能分析线段树与 ODT	36
5.3. 根号算法	37
5.3.1. 根号分治	37
5.3.2. 分块	37
5.3.3. 莫队	38
5.3.4. 莫队二次离线	38
5.4. 数据结构的常用维护方法	39
5.4.1. CDQ 分治	39
5.4.2. 线段树分治	39
5.4.3. 整体二分	39
5.4.4. 猫树分治	40
5.4.5. 二进制分组	40

6. DP 方向	40
6.1. DP 的单调性优化	40
6.1.1. 单调队列优化 DP	40
6.1.2. 斜率优化	41
6.1.3. 决策单调性优化 DP	41
6.1.4. wqs 二分	42
6.2. 矩阵优化 DP	42
6.2.1. 矩阵优化线性变换式 DP	42
6.2.2. 动态 DP	43
6.3. 其他 DP 有关的 trick	44
6.3.1. DP 套 DP	44
6.3.2. 整体 DP 与长链剖分优化 DP	44
6.3.3. 轮廓线 DP 与插头 DP	44
7. 数论方向	45
7.1. gcd 相关基础知识	45
7.1.1. 斐蜀定理	45
7.1.2. exgcd	45
7.1.3. 逆元	45
7.2. 欧拉函数与欧拉定理	45
7.2.1. 欧拉函数	46
7.2.2. 欧拉定理	46
7.3. 离散对数	46
7.3.1. BSGS 算法	46
7.3.2. 扩展 BSGS 算法	47
7.4. 线性同余方程组	47
7.4.1. 中国剩余定理 CRT	47
7.4.2. 扩展中国剩余定理 exCRT	47
7.5. Lucas 定理相关	47
7.5.1. Lucas 定理	47
7.5.2. 扩展 Lucas 定理	48
7.5.3. Kummer 定理	48
7.6. 阶与原根	48
7.6.1. 阶	48

7.6.2. 原根	49
7.7. 数论函数与莫比乌斯反演	49
7.7.1. 莫比乌斯函数	49
7.7.2. 数论函数	49
7.7.3. 狄利克雷卷积	50
7.7.4. 整除分块	50
7.7.5. 莫比乌斯反演	50
7.7.6. $f * I$ 的快速求法	51
7.8. 积性函数前缀和的求法	51
7.8.1. 线性筛	51
7.8.2. 杜教筛	52
7.8.3. Powerful Number 筛	52
8. 计算几何方向	53
8.1. 向量与向量的运算	53
8.2. 凸包	53
8.3. 最小圆覆盖	54
8.4. 半平面交	54
9. 网络流与二分图方向 (考察概率 30%)	55
9.1. 网络流与费用流	55
9.1.1. 最大流的求法	55
9.1.2. 最小费用最大流的求法	56
9.2. 上下界网络流	57
9.3. 二分图	57
9.4. 常见模型	58

1. 字符串方向

1.1. KMP

KMP 算法一般用于解决以下两类问题：

- 给定字符串 s ，对于 s 的每个前缀求其最长 border kmp_i 。
- 给定字符串 a, b ，对于每个 $1 \leq i \leq |a|$ ，求最大的 j 使 $a[i-j+1..i] = b[|b|-j+1..|b|]$ 。

求解前者的方法就是维护一个指针 j ，初始为 kmp_{i-1} ，然后不断跳 kmp 直到 $s_{j+1} = s_i$ 为止。求解后者的方法也类似，每插入一个字符就跳 $fail$ 直到当前指针的下一个位置与待插入字符相同。由于每次插入字符最多使指针在 KMP 树上的深度加 1，所以复杂度线性。

1.2. 2 算法

2 算法可以做以下事情：

- 给定字符串 s ，对于每个 $i = 2 \dots |s|$ 求 $s[i \dots |s|]$ 与 s 的 LCP z_i 。

求解方法类似于暴力，但是与暴力不同的地方在于，假设我们已经求完了 $z_2 \sim z_i$ ，我们实时维护使 $z_j + j$ 取到最大值的 j ，设其为 x ，那么我们求一个 z_i 时，如果 $i < z_x + x$ ，我们就以 $\min(z_{i-x+1}, z_x + x - i)$ 为 z_i 的初始值。不难发现， $z_x + x$ 是单调递增的，因此该算法复杂度也是 $\mathcal{O}(|s|)$ 的。

1.3. AC 自动机

可以看作 KMP 的扩展，只不过复杂度需要乘一个 Σ 。

如果模式串只有一个，那 KMP 显然是没有问题的，但是如果模式串有多个直接 KMP 显然复杂度多个 $n \cdot |t|$ 。考虑建出这些模式串的 trie 树，对于 trie 树上的每个节点 x 定义 $fail_x$ 表示 trie 树上深度最深的节点 y 满足 $root \rightarrow y$ 组成的字符串是 $root \rightarrow x$ 的一个后缀。求解 fail 的过程就从根节点开始 BFS，类比 KMP，求解 $fail_x$ 就从 fa_x 开始不断跳 fail 直到存在一个等于 x 与 fa_x 之间的字符的出边，移到对应节点即可。

事实上关于这个 $fail$ 还有更深层次的理解，考虑文本串与这一堆模式串匹配的过程，那么在任意时刻肯定存在一个最深的点 x 使得当前字符串的后 dep_x 个节点刚好对应节点 x ，显然在匹配的时候我们不用关心 dep_x 个字符往前的内容，因为它不可能走到一个模式串的位置，因此我们就设 x 为文本串匹配的状态。那么加入一个字符 c 的时我们会转移到一个新的状态 nw ，那么如果 x 存在 c 的出边，显然 nw 就是对应的儿子，否则我们就要退一步，但是为了避免错过可能的状态，我们只能退到 trie 树上能表示出来的最深的点，也就是 $fail_x$ ，如此跳下去直到能接上字符 c 为止。

AC 自动机是离线算法，即，不能支持在某个文本串后面加入字符后动态维护 $fail$ 的变化，如果碰到类似的题需要时间轴分块/二进制分组。

将 $x \rightarrow fail_x$ 连边，会得到一棵树称为 fail 树，那么求模式串在给定文本串的总出现次数，等价于对于文本串每个前缀，在 AC 自动机 trie 图上定位到其位置，然后将所以模式串的终止节点标记为关键节点，统计 fail 树上该节点到根路径上关键节点个数求和，因此 AC 自动机也常与树论结合。

1.4. SA

考虑倍增：我们考虑维护一个长度 len ，初始 $len = 1$ ，然后每一轮令 len 乘 2 并通过形如 $s[i \dots i + len - 1]$ 的 n 个子串排序后的结果来得到形如 $s[i \dots i + 2len - 1]$ 排序后的结果。具体方法就是，设 rk_i 表示 $s[i \dots i + len - 1]$ 的排名，那么等价于将形如 (rk_i, rk_{i+len}) 当作二元组排序，由于值域只有 $\mathcal{O}(n)$ ，排序可以桶排。注意相同的二元组的排名也应相同，其他方面在实现上还有一些注意点，譬如二元组的第二维其实并不需要桶排，可以直接 $\mathcal{O}(n)$ 地扫一遍得到其大小关系，然后再对第一维桶排即可，时间复杂度线性对数。

后缀数组可以延申出一套定理，在下面的讨论中我们设 sa_i 表示排名为 i 的后缀是谁， rk_i

表示 $[i...n]$ 的排名, 那么我们假设 $ht_i = \text{LCP}(s[sa_i...n], s[sa_{i+1}...n])$, 那么对于排名 x, y 的后缀 $sa_x, sa_y (x < y)$, 有定理: $\text{LCP}(s[sa_x...n], s[sa_y...n]) = \min_{i=x}^{y-1} ht_i$, 证明略。这是一个 RMinQ 的形式, 因此 SA 常与 DS 结合。那么如何求 ht_i 呢? 令 $h_i = ht_{rk_i}$, 那么有定义 $h_i \geq h_{i-1} - 1$, 因此从下标 1 枚举到 n 顺着扫一遍就好了。

这也就是为什么 SA 题目一般 getsa, getht, buildst, queryst 一遍写过去 (

1.5. Manacher

Manacher 算法一般用于求解一个字符串最长回文子串长度。

考虑先对字符串做一个变换: 对于字符串 $s_1s_2\cdots s_n$ 我们构造 $t = |s_1|s_2|\cdots|s_n|$, 即在相邻两个字符之间加入分隔符。这样可以避免对字符串长度的奇偶性进行分类讨论。

考虑设 len_i 表示以 i 为中心的回文串半径的最大值, 那么不难发现一个字符串最长回文子串的长度就是 $\max_{i=1}^{|t|} len_i - 1$ 。接下来考虑如何求 len 数组。类比 Z 算法, 我们从左到右求这些 len , 定义一个位置的回文串 box 为 $[i-len_i+1, i+len_i-1]$, 那么我们实时维护右端点最大的回文串 box, 然后求解 len_i 的时候, 假设右端点取到最大值的 box 为 $[x-len_x+1, x+len_x-1]$, 那么如果 $i \in [x-len_x+1, x+len_x-1]$, 我们就令 len_i 的初始值为 $\min(len_{2x-i}, x+len_x-i)$, 然后开始扩展即可, 这样每多匹配一格右端点就会加一, 复杂度线性。

推论: 一个字符串本质不同回文子串个数是线性的, 因此碰到回文串有关问题可以考虑这些性质。

1.6. SAM

1.6.1. SAM 与后缀树

对于给定字符串 s , SAM 是一个能够识别其所有子串的自动机。更具体地, 从初始状态到所有状态的路径都是 s 的一个子串, 并且 s 的所有子串都可以通过初始状态到某个状态的某条路径表示出来。显然 SAM 有 n^2 的建法, 太逊了, 考虑如何 $O(n)$ 地建 SAM。先抛出一些定义:

- $\text{endpos}(t)$, t 在 s 中所有出现次数的结束位置的集合。
- $\text{shortest}(t)$, 所有 endpos 与 t 相同的字符串中长度中长度最小的那个。
- $\text{longest}(t)$, 所有 endpos 与 t 相同的字符串中长度中长度最大的那个。
- $\text{minlen}(t)$, $\text{shortest}(t)$ 的长度。
- $\text{maxlen}(t)$, $\text{longest}(t)$ 的长度。

有了这些定义以后我们可以直观地想到: 将所有 endpos 相同的状态缩成一个等价类, 那么感性上理解这些等价类个数不会太多, 因此我们考虑将每个等价类看作一个状态。在进行接下来的讨论之前先抛出一些引理:

1. 任意两字符串的 endpos 要么包含要么不交, 读者自证不难。
2. endpos 相同的字符串的长度构成一段连续的区间, 且较短者永远是较长者的后缀, 读者自证不难。
3. 状态数的上界是 $2n-1$, 读者自证不难 (显然将区间的包含关系连边会连出一棵树, 其叶子节点上界是 n , 因此总结点数上界是 $2n-1$)

引理三为我们将后缀自动机复杂度降到线性埋下了基础。接下来引入另一个定义：

- 定义一个状态的后缀链接表示，该状态所表示的字符串中，最长的 endpos 不等于该状态的后缀表示的状态的后缀所表示的状态，下文中记作 $\text{link}(p)$ 。

定义一个字符串的后缀树为 $i \rightarrow \text{link}(i)$ 连边后形成的树（为什么是树呢？因为显然在一个字符串开头砍掉若干个字符后它的 endpos 集合大小肯定是单调不降的，根据引理 1，这张图必然不会成环，因此它是树）

当然，由 link 的定义也可以直接得出一些结论：

- $\text{maxlen}(\text{link}(i)) + 1 = \text{minlen}(i)$ 。
- 后缀树上从 i 到根节点的路径本质上是从开头删去字符。
- 对于一个状态 i ，它在后缀树上到根的路径上所有点的 $[\text{minlen}, \text{maxlen}]$ 不交且并集为 $[0, \text{maxlen}(i)]$ 。

上面的讨论是基于后缀树的一些理论，那么我们又该如何将这套理论与自动机的理论结合在一起呢？由于我们将 edp 相同的缩成了一类，所以有关状态和转移的定义也需做相应的修改：

- 状态：SAM 上的一个状态表示一个 edp 相同的等价类，即知道当前所在自动机上的节点，就可以知道当前字符串的 edp 。
- 初始状态：空串所表示的状态。
- 转移：对于一个状态 x ，其转移 $\delta(x, c)$ 表示对于该等价类中的所有字符串，在其末尾加上 c 之后能够到达的状态，根据该等价类所有字符串相同可知最多只会到达一个状态， $\delta(x, c)$ 也就唯一表示了这个状态。

这样我们可以知道，对于一个字符串表示的状态，在其末尾插入字符可以视作在自动机上转移，在其开头插入字符可以视作在后缀树上向其儿子转移。

1.6.2. SAM 的建立

现在我们已经知道了后缀树与后缀自动机的联系，下面我们要知道如何构建后缀自动机。

假设现在我们已经知道了当前字符串 $s[1\dots n]$ 的后缀自动机，我们要在后面 append 某个字符 c ，考虑其变化。

首先我们显然可以在每插入一个字符的时候就维护整个串表示的状态 x ，首先我们添加转移 $\delta(x, c)$ ，并令 y 为转移到的新节点。考虑加入这个节点会多出哪些变化。显然存在一个最长的后缀 $s[i\dots n]$ 满足 $s[i\dots n] + c$ 还是 $s[1\dots n]$ 的一个子串，对于比这个后缀更长的后缀表示的状态 t ，我们添加转移 $\delta(t, c) = y$ ，这一部分暴力跳即可。特判掉不存在这样的后缀的情况，此时直接令 $\text{link}(y)$ 为根节点并返回。我们假设 $s[i\dots n]$ 表示的状态为 p ， $\delta(p, c)$ 表示的状态为 q 。

显然，根据 edp 的定义， $\text{maxlen}(q) \geq \text{maxlen}(p) + 1$ ，此时我们分两种情况讨论：

- $\text{maxlen}(q) = \text{maxlen}(p) + 1$ ，此时并不会多出新的等价类，对于 q 在后缀树上的祖先节点，它们的 edp 中会多一个 $n + 1$ ，其余节点的 edp 并不会改变，而根据定义 y 的后缀链接就

是 q ，因此我们直接令 $\text{link}(y) = q$ 然后返回即可。

- 否则我们发现 q 等价类所包含的字符串可以分为两类：一类长度 $\leq \text{maxlen}(p) + 1$ ，一类长度 $> \text{maxlen}(p) + 1$ ，二者几乎相同：由于在它们后面插入任何一个字符以后，所得的字符串的 edp 都不会包含 $n + 1$ ，所以它们在 SAM 上的转移也完全一致，唯独不同之处在于前者的 edp 包含 $n + 1$ ，而后者不包含，所以原来的等价类需要分裂成两个等价类。那么分裂成两个等价类又会对其他节点的转移产生怎样的影响呢？我们令前者的状态为 cl ，后者保持它原来的标号 q 不变，那么该自动机的 δ 和 link 会产生如下变化：
 - $\text{link}(q) = \text{link}(y) = cl$ ， $\text{link}(cl)$ 则等于原先的 $\text{link}(q)$ ，这是显然的。
 - 对于原先 $\text{link}(t) = q$ 的状态 t ，它们现在的 link 肯定还是 q 保持不变。因此除了 $\text{link}(q), \text{link}(y), \text{link}(cl)$ 其他点的 link 均不会发生变化。
 - 对于 p 在后缀树上到根节点的那段路径上 $\delta(t, c) = q$ 的状态，在这些等价类的所有字符串中加入 c 后它们的 edp 都会包含 $n + 1$ ，因此它们的 $\delta(t, c)$ 都会改为 cl ，而对于其他包含 c 的转移边且 $\delta(t, c) = q$ 的点，它们在加入字符 c 后的 edp 都不会包含 $n + 1$ ，因此它们的 $\delta(t, c)$ 都不会发生变化。这样我们就直接从 p 开始跳 link 暴力修改它们的 $\delta(t, c)$ 即可。

时间复杂度可以被证明是线性的。但是限于篇幅原因，这里不证明。

1.6.3. 一些后缀自动机的基本技巧

- 一个字符串的 endpos 集合：如果表示 $s[1...i]$ 的状态在该字符串所表示状态的后缀树的子树内，那么该字符串的 endpos 则包含 i ，否则该字符串的 endpos 不包含 i 。可以使用线段树合并维护。
- s 的某个子串 $[l, r]$ 所表示的状态：找到 $s[1...r]$ 表示的状态，倍增找到最浅的 $\text{maxlen} \geq r - l + 1$ 的点。
- 该图父亲的 len 值一定比儿子小，因此可以对 len 进行桶排求解 DFS 的顺序，类比 DFS 序倒着遍历在某些卡常题中的作用。

至于 PAM 什么的感觉 NOI 考的频率 $< \epsilon$ 所以也不准备复习了，毕竟早忘了（

2. 图论方向 (考察概率 100%)

2.1. 最短路

为什么要写呢？因为感觉每年都有人因为这种东西写挂寄掉/cf

2.1.1. Bellman ford

显然如果存在最短路最短路边数必然 $\leq n$ ，因此更新 n 轮，每轮对于每条边 (u, v, w) 用 $\text{dis}_u + w$ 更新 dis_v ，复杂度 nm ，可以处理负权边 + 负环。时间复杂度 nm 。

2.1.2. Dijkstra

如果边权都是正数，那么每次 dis 最小的不会再更新了，因此直接取出最小的去更新别的点即可。注意加 visit 优化，否则复杂度平方 $\log$ 。不能处理负权边。不能处理负环。时间复杂度

$m \log m$ 。

2.1.3. SPFA

维护一个队列，每次取出队首元素，如果一个点被松弛，并且不在队列里，我们就把它压入队列。如果一个点松弛次数 $> n$ 说明有负环。可以处理负权边。可以处理负环。时间复杂度 nm 。

2.1.4. Floyd

先考虑一个 DP。 $dp_{k,i,j}$ 表示只经过 $1 \sim k$ 的 $i \rightarrow j$ 的最短路。容易转移。容易发现第一维可以省去，可以处理负权边。可以处理负环。注意顺序先 k ，然后 i ，最后 j 。时间复杂度 n^3 。

Floyd 可以使用 bitset 将传递闭包复杂度优化至 $\frac{n^3}{\omega}$ 。

2.2. 最小生成树

2.2.1. Prim

大致思想：维护一个集合 S 表示目前已经构建最小生成树的部分，每次考虑找出连接集合内一个点和集合外一个点，且权值最小的边，将这条边所连接的不在集合中的点加入集合。时间复杂度 n^2 ，堆优化 $m \log m$ 。有时候后者不如前者。

2.2.2. Kruskal

将所有边按照权值从小到大排序，然后依次枚举每条边，如果它的两个端点不在同一连通块中就将其加入边集。时间复杂度 $m \log m$ ，瓶颈在排序。

2.2.3. Boruvka

相较于前两者算是一个较为复杂的算法了。一般用于解决图近似完全图但边权有一定性质的题。

首先先考虑令每个点单独一个连通块。然后考虑一轮一轮地进行合并操作，每一轮我们考察所有连通块，对于每个连通块我们找到从这个连通块中任意一个点连向其他连通块中的边中权值最大的，这样总共有 连通块个数 条边（其中有些边可能有重复），并查集合并即可。显然每轮连通块个数都会变为原先的除以二上取整，因此轮数是对数级别的。一般“找连向其他连通块中的边中权值最大的”这项操作需使用数据结构维护。

2.3. 差分约束

给定若干个形如 $x_a - x_b \geq c$ ($\leq$ 也行) 的限制，要求出一组符合条件的解。注意到这东西有点像最短路中的三角不等式 $dis_v \leq dis_u + w$ ，因此考虑将其转化为最短路建模。对于每组形如 $x_a - x_b \leq c$ 的限制，我们就连一条 $b \rightarrow a$ 权值为 c 的边。对于 $x_a - x_b \geq c$ 我们将其转化为 $x_b - x_a \leq -c$ ，即连 $a \rightarrow b$ 权值 $-c$ 的边，然后跑 SPFA/Floyd 即可。如果给定下界，那么最短路就是差分约束字典序最小的解。差分约束一般用于解决非策略性构造问题，即如果一个构造题无法用常规的构造性策略解决，并且其能够被归约到决策一个序列 x_i 的取值使其满足某写形如 $x_a - x_b \geq c$ 的限制的题目。

2.4. 2-SAT

一个同样可以用来解决非策略性构造题的算法。

给定若干个布尔型变量 x_i ，有若干个形如“如果 A 则 B ”的命题，其中 A, B 均为形如 x_i 或者 $\neg x_i$ 的表达式，或者满足 x_i 必须为真 / 假的条件。考虑对所有 x_i 和 $\neg x_i$ 的布尔型变量均新建一个节点，对于每组限制连边 $A \rightarrow B$ 表示 A 可以推导到 B ，以及其逆否命题 $\neg B \rightarrow \neg A$ 。如果强制要求 x_i 为真则连边 $\neg x_i \rightarrow x_i$ ，这样则 ban 掉了 $\neg x_i$ ，强制要求 x_i 为假的情况也同理。然后跑强连通分量缩点，如果存在 i 使得 x_i 和 $\neg x_i$ 在同一强连通分量中，那么显然无解，否则由于 tarjan 缩点编号按照拓扑序逆序排序，如果 x_i 所在强连通分量小于 $\neg x_i$ ，那么则说明在我们构造出的解中 $x_i = 0$ ，否则 $x_i = 1$ 。

2-SAT 一般用于解决选某个子集，构造方案或判断有解性（而不是计数，因为 2-SAT 计数是 NPC 问题），且有若干形如若 A 则 B 的限制的题目。

当然如果要求字典序最小的解，则只能套路化按位贪心 $n(n+m)$ 或者 $\frac{n^3}{\omega}$ ，如果要做到更优复杂度则可能需要分析一些性质？

2.5. 同余最短路

给定 n 个值域很小的数 $a_1 \sim a_n$ ，要求 $1 \sim m$ 中有多少个数能够表示为 $\{a_n\}$ 的线性组合。

不妨假设 $\{a_n\}$ 单调递增。一个浅显的性质是如果 x 能表示为 $\{a_n\}$ 的线性组合，那么 $x + a_1$ 也行，因此我们考虑设 dp_i 表示最小的 x 满足 $x \equiv i \pmod{a_1}$ 且 x 可以表示为 $\{a_n\}$ 的线性组合，那么对于一个 a_i ，有 $dp_j + a_i \rightarrow dp_{(j+a_i) \bmod a_1}$ 。直接 dijkstra 即可。

时间复杂度 $na_1 \log(na_1)$ ，即便 $a_2 \sim a_n$ 都很大，只要 a_1 很小，都可以同余最短路（分析性质？）

2.6. 欧拉回路

对于无向图而言，定义其欧拉回路为一条经过每条无向边恰好一次最后回到起点的路径，欧拉路径为一条经过每条边恰好一次且起点与终点不重合的路径。对于有向图也有类似的定义。那么有如下结论：

- 对于无向图，其有欧拉回路当且仅当去掉孤立点后其是连通图，且每个点度数都是偶数。
- 对于无向图，其有欧拉路径当且仅当去掉孤立点后其是连通图，且恰有两个点度数是奇数。
- 对于有向图，其有欧拉回路当且仅当去掉孤立点后其基图是连通图，且每个点的入度都等于出度。
- 对于有向图，其有欧拉回路当且仅当去掉孤立点后其基图是连通图，且恰有一个点入度等于出度减一，也恰有一个点入度等于出度加一。

必要性是容易证明的，以前者为例，显然每个点在每次经过的时候都会有一个前驱边和一个后继边，它们显然都是互不相同的，因此与每个点相连的边的个数必须是偶数，否则就 GG 了。

充分性则可以通过构造来证明。考虑以下伪代码：

```
function DFS(u):
```

```
for every edge(u, v) that has not been visited:
    DFS(v);
insert u into the path;
```

考虑证明上面流程的正确性。我们试图证明，除了在最后一个点（也就是最后一次 DFS 起点时），其他情况下如果回溯，那么说明起点的所有出边均已被遍历。显然如果我们还没有进行过回溯，当我们 DFS 到一个点 u 时，如果 u 不是起点，那么我们考察与每个点的被访问过的边的数量，可以得出除了 u 和起点是奇数外，其他点都是偶数。也就是说，如果 u 不是起点，那么此时 u 必然还有路可走，会继续向下 DFS，不会发生回溯，唯一可能发生回溯事件的情况在于 u 回到了起点，并且现在 u 的出边已经访问完了。此时，显然剩余未访问的边集是若干个欧拉子图的并。那么如果我们回溯到一个还有边没被访问过的点怎么办呢？不难发现，我们会继续在这个点所在连通块中，把这个点当作起点继续进行欧拉回路。而由于我们到访问结束回溯时才将点加入路径，因此第一个被加入路径的肯定是回溯时的起点，因此我们感性理解，最后 path 得到的肯定是欧拉回路的倒序，对每个子图进行归纳即可得到结论。

还有一个需要注意的点是，我们遍历与每个点相连的边时，如果一条边被访问过，那么我们就不需要再访问它了，因此如果使用链式前向星存图，需要将枚举变量 e 传引用，否则复杂度会退化到平方。

欧拉回路时间复杂度为线性。有的题，如果要求“给每条边染色”，并要求“与每个点相连的两种颜色个数差绝对值 ≤ 1 ”这样的字眼，要立马反应过来，是建虚点后欧拉回路。

2.7. 无/有向图连通性

2.7.1 Tarjan

给定一张有向图，要求其所有强连通分量，即所有极大的点集，满足其中所有点之间两两均可达。

考虑 DFS，DFS 时维护两个数组 dfn_x 和 low_x ，和一个栈表示现在 DFS 过但还没有确定强连通分量的点， dfn_x 表示 x 的时间戳， low_x 表示 x 能到达的在栈中的点中，时间戳最小的。DFS 时就遍历与 x 相连的边 (x, y) ：

- 如果 y 没有被访问过，即 $dfn_y = 0$ ，那么 DFS 它，并令 low_x 对 low_y 取 min。
- 如果 y 被 DFS 过但是不在栈中，那么 skip 掉。
- 否则令 low_x 对 dfn_y 取 min。

然后当一个点 $dfn_x = low_x$ 时就弹掉 x 以上的部分，令它们为一个强连通分量即可。正确性显然，如果 $dfn_x = low_x$ 则表明该点集的连通性已经封闭了，无法再加入其他点使其还是一个强连通分量。

顺带着提一下 tarjan 过程的三种边：

- 树边：对应上述第一种情况的边 (x, y) ，即 DFS 树上的边。
- 横叉边：对应上述第二种情况的边 (x, y) ，横叉边的两个端点永远不属于同一个强连通分量，横叉边永远不成环。
- 返祖边：对应上述第三种情况的边 (x, y) ，返祖边会成环。

tarjan 时间复杂度 $O(n + m)$ 。

2.7.2 VBCC/EBCC

Tarjan 是针对有向图的连通性而言的，而 VBCC 和 EBCC 则是针对无向图的连通性而言的。它们的定义如下：

- VBCC：极大的满足任意两点间都有至少两条不在除了起终点处相交的路径，注意两个点一条边的图也是 VBCC。
- EBCC：极大的满足任意两点间都有至少两条不经过同一条边的路径。

VBCC 的性质是内部不存在割点（即去掉后图不连通的点），EBCC 的性质是内部不存在割边（即去掉后图不连通的边）。也就是说 VBCC 由割点相连，EBCC 由割边相连。考虑如何找出割点和割边，魔改上面的 tarjan 算法可以得到：

- 一个点是割点，当且仅当其不是该连通块中第一个 DFS 的节点且其所有子节点的 low 都 $\geq$ 该点的 dfn ，或者其是该连通块中第一个 DFS 的节点且其有超过两个儿子满足遍历该点与儿子的边时，该儿子没有被访问过。
- 一条边 $(u, v)(dfn_u < dfn_v)$ 是割边，当且仅当 $low_v > dfn_u$ 。

显然求出割边后直接去掉所有割边后 DFS 一遍整张图，它的每个连通块都是一个独立的 EBCC。这样我们会求解 EBCC 了。但是我们求出割点后不能简单地求解 VBCC，甚至更麻烦的事情是，每个点可能同时处于多个 VBCC 中，这时我们就需要一个更加强大的工具：圆方树。

2.7.3 圆方树

圆方树，说白了就是一个点数上界为 $2n$ 的树，其中圆点为原图中的点，方点为每个 VBCC 表示的点。对于每个 VBCC，都连一条从对应方点到该 VBCC 中所有圆点的边。很明显如果原图是连通图，那么连出来的也是树。

首先考虑圆方树的建法，还是考虑魔改 tarjan。当我们 DFS 到一个节点 x 时，我们肯定还是遍历其所有邻居 y ，如果没访问过则 DFS y ，否则令 low_x 对 low_y 取 $\min$ ，如果 $low_y \geq dfn_x$ ，那么说明 y 子树内的点加上 x 形成了一个点双，此时我们就把 y 子树内的部分弹掉，然后额外添加新点与 x 之间的边即可。

圆方树的好处是易于将 x, y 之间所有简单路径的并转化为树上路径，这样信息也方便维护：方点象征整个点双，维护整体信息，圆点象征树上的某个点，维护单点信息，这样一来，维护譬如所有简单路径权值 $\min$ 这样的问题就变得很容易了：根据 VBCC 的性质，任意两个点之间的必经之点就是圆方树上它们之间路径上的点，可能经过的点就是它们路径上所有方点的邻居的并，因此我们直接方点维护所有与其相邻的圆点的权值的 $\min$ ，然后查询路径最小值即可。注意，修改一个圆点权值时不能遍历其所有邻居，正确方法是只修改其父亲的权值，查询时如果 LCA 是方点就令权值与其父亲权值取 $\min$ 。

2.8. 数据结构优化建图

有的时候，暴力建图边数过多，这时候就需要数据结构优化建图了。

2.8.1. 线段树优化建图

可以解决形如一个点像一个区间连边，或者一个区间向一个点连边，或者一个区间向一个区间的情形。

以第一种为例，建一棵线段树，父亲向儿子都连边，当我们从 x 向 $[l, r]$ 连边时，就将 $[l, r]$ 拆分成 $\log$ 个子区间，然后从 x 向这些子区间连边即可。

树剖优化建图、二维线段树优化建图等都是同理的，这里不再赘述。

2.8.2. 前后缀优化建图

如果上面的区间是一段前后缀，也可以采用前后缀优化建图。

具体来说就是如果要向 $[1, x]$ 连边，就建一排虚点 $p_1 \sim p_n$ ，然后建边 $p_i \rightarrow p_{i-1}, p_i \rightarrow i$ ，这样只要向 p_x 连边就等价于向 $[1, x]$ 连边。

2.8.3. 堆优化存边

对于一些最短路问题，如果点数很小，但边数很多且都是由一个点向一个区间之类的东西连边，可以考虑堆优化存边。

在 dijkstra 的堆里我们不存点，改存边。以一个点向一段区间连边为例，比方说我们现在有一条边 $x \rightarrow [l, r]$ 权值为 w ，我们更新完 x 的 dis 以后就往堆里扔一个三元组 $(dis_x + w, l, r)$ 。然后每次取出堆顶三元组 (d, l, r) 时，就检查 $[l, r]$ 中 dis 没有更新过的点，将它们的 dis 赋为 d 。可以使用 set，更好的办法是并查集（区间删点问题经典 $n\alpha(n)$ 做法），具体方法就是如果一个点 x 被删去我们就合并 x 和 $x+1$ 所在的集合，这样一个点下一个没被删去的点就是它所在的集合中最靠右的点。

3. 树论方向

按照随机顺序复习算法。

3.1. LCA

太基础了。一般有倍增法，树剖法和 ST 表法。这里讲下 DFS 序求 LCA 方法，是一个很好的小常数 $O(1)$ LCA 方法，六边形欧拉序 LCA。大概就是求出每个点的 DFS 序 dfn_x ，然后 ST 表下标为 dfn_x 的位置存储其父亲的深度和编号，然后查询 u, v 的 LCA 先特判掉 $u = v$ ，然后假设 $dfn_u < dfn_v$ 就查询 $[dfn_u + 1, dfn_v]$ 的最小值即可（注意左端点要加一）

与 LCA 有关的是树上差分，这里稍微提一句，以免考场降至，就是 u, v 的路径等于 $root \rightarrow u$ 加上 $root \rightarrow v$ 减去 $root \rightarrow LCA(u, v)$ 减去 $root \rightarrow fa_{LCA(u, v)}$ ，这样路径操作可以转化为四次单点操作，单点查询可以转化为子树查询。

3.2. 树链剖分

对每个点令其 siz 最大的儿子为重儿子。令 $x \rightarrow wson_x$ 为重边，其余边为轻边，重边形成的链为重链。那么最重要的性质是一个点到根节点路径上最多有 $\log$ 条轻边。证明是显然的。

树链剖分基础应用都太容易了这里不再赘述。比较困难的是路径邻域修改问题，即给你一条

路径，要你对这条路径的邻域进行某种操作。具体维护方法是，对于重边，暴力使用线段树修改一条链，对于轻边，由于路径上最多有 $\log n$ 个轻边，所有可以在这些轻边的父亲处暴力维护所有轻儿子的答案。查询也是类似的，重边的贡献使用线段树维护，轻边的贡献暴力加上。如果不是边权而是点权也同样处理，将点分为“其与父亲的边是重边”和“其与父亲的边是轻边”处理。

3.3. Kruskal 重构树

Kruskal 重构树与 Kruskal 唯一的区别在于，每次合并一条边时建一个虚点，并连一条虚点到两个端点所在连通块的边。并令该点的点权为这条边的边权。以最小生成树为例，Kruskal 重构树有以下性质：

- 两点在最小生成树上路径的最大值，等于它们 Kruskal 重构树上 LCA 的权值。
- Kruskal 过程中任何时刻的任何一个连通块，都可以用 Kruskal 重构树上一个子树来刻画。
- 一个点经过 $\leq d$ 的权值能够到达的点，等于这个点深度最浅的权值 $\leq d$ 的祖先子树内的点。

如果是点权，Kruskal 重构树还有多叉的建法，即按照点权从小到大加入点，加入一个点时遍历其所有邻居，如果邻居点权比自己点权小就连一条自己到邻居的边。

配合倍增，Kruskal 可以解决很多带有“最小瓶颈路”、“经过权值不超过 d 的点 / 边”、“路径上点 / 边权最大值的题目，碰到这些字眼需要格外注意。

3.4. 虚树

给定树上若干个关键点，有的时候，我们并不关心这些点链上其他非关键点的信息，而只关心关键点之间的相对顺序关系。此时我们就可以使用虚树建出一个和原树形态相同的缩小版树。

具体建立方法是，我们先将点集按 dfn 排序。然后考虑维护一个栈表示目前最右链上的节点，栈顶节点深度最大，栈底节点深度最小。加入一个点 x 时，如果栈为空则直接入栈，否则我们考虑以下流程：

- 弹出栈顶元素直到从上往下数第二个点的深度 $\leq$ LCA 的深度，每弹掉一个 $stk[tp]$ ，就在虚树上连边 $stk[tp-1] \rightarrow stk[tp]$ 。
- 如果栈中剩余至少一个元素且栈顶元素比 LCA 深度大，那么连边 $LCA \rightarrow stk[tp]$ 并弹出栈顶元素。
- 如果此时栈为空，或者栈顶元素不等于 LCA，则将 LCA 入栈。
- 将 x 入栈。

在所有点插入完成以后，还需要将栈情况，即，弹出栈顶元素知道栈大小为 1，每弹一个元素都在虚树上连边 $stk[tp-1] \rightarrow stk[tp]$ 。

对 S 建立虚树建立的复杂度是 $|S| \log n$ 的，瓶颈在于排序和 LCA。

值得注意的是，有时候虽然涉及到虚树，但有一些我们不用建出虚树，直接算就可以算出来的简洁式子，譬如假设点集 S 按 DFS 序排序后的结果为 $p_1 \sim p_k$ ，那么虚树上所有边所对应

的链长和为 $\frac{1}{2}(\text{dis}(p_1, p_k) + \sum_{i=1}^{k-1} \text{dis}(p_i, p_{i+1}))$ ，原树中虚树所覆盖到的点的数量也就是其加 1，这可以作为线段树合并信息的方式。ZJOI2019 语言这题也就用到了这个套路。

3.5. 树上启发式算法

3.5.0 简介

所谓树上启发式算法，就是要对树上每个子树维护一些信息，并且两个子树的信息可以在 $O(\min(|A|, |B|))$ 的时间复杂度内完成。一般来说根据子树信息级别可以使用 dsu on tree、长链剖分、线段树合并、set 启发式合并等算法完成。一般树上启发式算法都要求离线。

3.5.1. dsu on tree (子树信息大小与 siz 同阶)

对树进行重链剖分，然后按照以下步骤进行 DFS：

- DFS 其轻儿子，将桶清空。
- DFS 其重儿子，不清空桶。
- 将 x 的贡献加入桶。
- 遍历 x 所有轻儿子，将轻子树的贡献加入桶。

时间复杂度与所有点到根路径上重链总数同阶。即插入的复杂度乘以 $n \log n$ 。

dsu on tree 能够使用的先决条件是插入单点复杂度不是很高，因此有的题合并复杂度较高可以使用该算法（至于合并复杂度低的，则可以使用线段树合并，下面会讲到）。

3.5.2. 长链剖分 (子树信息大小与 mxdep 同阶)

所谓长链剖分，就是定义每个点的深儿子 $dson$ 为 $mxdep$ 最大的节点，类比重链剖分，定义 $i \rightarrow dson_i$ 的边为长边，其他边为短边。长边组成的链为长链，那么有一个很显然的性质是所有长链和是线性的，这个性质有以下两个应用：

3.5.2.1. $O(n \log n) - O(1)$ 树上 k 级祖先。 先倍增预处理每个点向上跳 2^i 步能够到达的点，以及对于一个长链的链顶节点 x ，预处理出这个点向上跳 $0, 1, 2, \dots, mxdep_x$ 步的结果，以及沿着长链向下走 $0, 1, 2, \dots, mxdep_x$ 步的结果，然后考虑以下步骤：

- 求出最大的 b 使得 $2^b \leq k$ ，然后将 x 向上跳 2^b 步到达点 y ， k 对应减去 2^b 。
- 将 y 跳到其长链链顶节点 z ， k 对应减去 $dep_y - dep_z$ 。
- 如果 $k > 0$ 那么则返回预处理得到的 z 向上跳 k 步的点，否则返回预处理得到的 z 向下跳 $-k$ 步的点。

首先根据所有长链长度和是线性的，因此预处理复杂度也是线性的。而显然 $mxdep_z \geq dep_x - dep_z = 2^b + dep_y - dep_z$ ，而最终的 k 的绝对值肯定是 $\leq 2^b + dep_y - dep_z$ 的，因此最后一步的信息肯定在我们预处理范围内，这样可以实现 $O(1)$ 查询。

3.5.2.2. 优化与深度有关的 DP。 长链剖分也可以优化形如 “ $dp_{i,j}$ 表示 i 子树内深度为 j 的点的 xxx”，即所有深度相同的点贡献同一个状态。具体方法就是，先 DFS x 的深儿子，然后当前点的 DP 状态由深儿子继承而来，然后再遍历所有轻儿子 y ，将它们对 dp_x 的儿子加进来。

不难发现这样在长链中间，这条长链的贡献都是 $\mathcal{O}(1)$ 从其深儿子处继承来的，只有在链顶节点的父亲处需要花费【长链长度】的代价合并，因此总复杂度是线性的!! 1

这东西实现起来稍微有点方法，因为没法开临时变量存储这些 DP 值，一种解决方法是指针，但是个人偏好是使用一个 `vector<int>` 存它们，`vector` 从第一个到最后一个元素分别表示 $dp_{x, mxdep_x}, dp_{x, mxdep_x-1}, \dots, dp_{x, dep_x}$ （即顺序是反的），每次继承的时候直接 `swap` 两个 `vector` 即可，如果要更新 dp_{x, dep_x} 就直接在 x 的 `vector` 里 `push_back` 即可。显然任意时刻的空间开销都是线性。

3.5.3 线段树合并（维护的信息可通过线段树的合并操作得到）

对于两个以 x, y 为根的动态开点线段树，我们定义它们合并的结果如下：

- 如果 x, y 之一是空结点，那么返回另一个非空的节点（如果两个都是空结点则返回空）
- 否则新建节点，令新建的节点的左儿子为 x 的左儿子与 y 的左儿子合并后的节点，右儿子也同理。

显然，把 n 个形如一条链的线段树合并成一个完整的线段树最多新建 $n \log n$ 个节点。因为每新建一个节点，都意味着 x, y 已经没用了，总结点数会减一。而初始一共有 $n \log n$ ，减到最后最多也只会增加 $n \log n$ 个节点。

那么这东西又怎样和树论结合起来呢？

我们假设每个点上有一个信息，现在我们要将每个子树的信息合并起来，显然这是线段树合并能解决的，具体来说就是 DFS，每次 DFS 到一个结点时就枚举其儿子，DFS 该儿子然后令该节点上的线段树为其与其儿子的线段树合并后的结果。显然这样复杂度还是 $1 \log$ 。

线段树合并在某些时候可以替代 `dsu on tree` 和长链剖分，但是常数略大。什么情况下只能线段树合并呢？如果你得到子树信息之后还需要对其进行线段树操作，如线段树二分，那么则只能线段树合并。此外，线段树合并也可以优化整体 DP，譬如一些树形 DP，它的合并只是点值相加（不涉及到点值的平移或卷积等操作），并且我们在求得一个子树的 DP 之后还要对其整体进行修改，这时候也只能线段树合并。

3.5.4. set 启发式合并

在所有树上启发式算法中，最简单粗暴的方法莫过于直接用 `set` 存储每个子树信息，然后小集合合并到大集合上去，这样复杂度 $2 \log$ ，比前几个算法都劣，因此如果前几个算法可以解决那肯定优先使用前几个算法。但是有时候 `set` 启发式合并可以用来解决前几个算法无法解决的问题，具体情况具体分析吧。

3.6. 点分治与点分树

3.6.1. 点分治

类比序列分治可以解决数区间问题，使用点分治也可以解决一些路径计数问题（要求必须是无根树，否则点分治无法解决）。具体方法就是找到当前连通块的重心，然后计算跨重心的路径条数，然后去掉重心之后继续分治剩余的连通块。由于每次分治连通块大小减半，所以分治层数最多是 $\log$ 级别的。

点分治的时候有一个偷懒的地方，就是分治的时候不用重新 DFS 求出子树的 $totsiz$ ，可以直接调用上一层分治的 siz ，可以证明这样复杂度还是正确的，只不过分治层数不一定是严格 $\log$ 的，有可能会乘个常数，如果要开关于分治层数的数组需要注意这点。

点分治统计有一个小技巧，就是如果信息满足可减性，不一定要像 dsu on tree 那样两遍遍历子树并先算贡献后加点，可以考虑先忽略“属于分治重心的不同子树”这一条件，在 $O(\text{集合大小} \cdot \log^k)$ 的复杂度内计算任意两条路径拼起来的答案，再容斥掉属于同一连通块的答案。

当然点分治不只适用于路径统计，如果分析出了与重心有关的性质，也可以考虑点分治。一个非常经典的模型：单一重心移动模型。即我们要求一个点 x 满足一定条件，且我们可以在 $O(n)$ 时间内 check 一个点 y 是否合法，且如果不合法我们可以知道合法点在 y 的哪个方向，我们就可以使用点分治，具体方法是类似于移动棋子，先把棋子放在连通块重心处，然后 check 是否合法，如果合法则返回，否则继续递归剩余几个连通块。时间复杂度 $\log n \cdot O(\text{check})$ 。

3.6.2. 点分树

如果点分治要支持修改，显然需要将点分治过程记录下来，这时就要用到点分树了。考虑对于以 x 为根的连通块，我们假设删掉 x 后形成的那些连通块的重心分别是 $y_1, y_2, \dots, y_k$ ，那么我们就从 x 向每个 y_i 连边。可以发现点分树和原树关系非常小，但是点分树有以下性质：

- 点分树的深度和点分治层数同阶，是 $\log$ 级别的，这意味着我们可以在每个点处维护大小为其点分树上子树大小的信息，总信息量也只有单 $\log$ 。
- 任意两点 x, y 在点分树上的 LCA 一定在它们原树的路径上。

第二个性质启发我们令 z 为点分树上两点的 LCA，然后将 $dis(x, y)$ 拆成 $dis(x, z) + dis(y, z)$ ，然后修改时从 x 开始跳祖先在这些祖先的数据结构处修改对应的 $dis(x, z)$ ，查询时还是跳祖先，每遇到一个祖先 z 都查询 z 那些不同于 x 所在的子树内所有点的 $dis(y, z)$ 之和 sum 与 $dis(y, z)$ 个数 cnt ，那么贡献就是 $dis(x, z) \cdot cnt + dis(y, z)$ 。值得注意的是，为了容斥掉同一子树内的贡献，这边信息一般要满足可减性，此时我们在每个点处再维护一个数据结构，存储每个点到其点分树上的父亲的 dis ，然后在 z 在 x 方向上的儿子内查询对应的值，减掉即可。时间复杂度 $n \log n \cdot \text{数据结构的复杂度}$ 。一般数据结构选择 BIT/动态开点线段树。

4. 数学方向 (考察概率 70%)

4.1 组合数与组合恒等式

组合数是组合数学最基础的东西之一。

$\binom{n}{m}$ 表示从 n 个数中选 m 个数的方案数，其值等于 $\frac{n!}{m!(n-m)!}$ ，组合数同时还有递推式 $\binom{n}{m} = \binom{n-1}{m} + \binom{n-1}{m-1}$ 。

以下是一些比较基础且常用的组合恒等式，容易用组合意义证明：

- $k \binom{n}{k} = n \binom{n-1}{k-1}$ (吸收恒等式)。
- $\binom{a}{b} \binom{b}{c} = \binom{a}{c} \binom{a-c}{b-c}$ (吸收恒等式的扩展版本)

- $\sum_{i=0}^a \binom{a}{i} \cdot \binom{b}{y-i} = \binom{a+b}{y}$ (范德蒙德卷积)
- $\sum_{i=0}^n \binom{n}{i}^2 = \binom{2n}{n}$ (范德蒙德卷积特例)
- $\sum_{i=0}^n \binom{n}{i} \cdot a^i \cdot b^{n-i} = (a+b)^n$ (二项式定理, 后面的 b 有的时候会等于 ± 1 , 有时需要把这里的 ± 1 补上)
- $\sum_{i=0}^b \binom{a+i}{a} = \binom{a+b+1}{a+1}$ (杨辉三角形从对角线上某个点开始一列的和, 等于其右下角的元素)

对于有些式子, 其是可以直接用组合恒等式化简的, 但是有些式子则不一定, 这时候有两种方法:

- 递推。即把组合数裂开, 看看可不可以将其化简为类似形式, 但是 n 更小的式子, 如果可以则可以进行递推求解。此种方法一般比较无脑。
- 找组合意义。然后构造双射或者转化形式之类的方法求解。如果比较擅长构造双射则可以考虑这种方法。

4.2. 容斥原理

容斥原理一般有两种用途:

- 通过计算交的大小来计算并的大小。
- 通过计算“包含某个子集的符合 xxx 要求的集合个数”, 来计算“不包含某个集合中任何一个元素的符合 xxx 要求的集合个数”。

前者是容斥原理最基本的用途, 因为

$$\bigcup_{i=1}^n S_i = \sum_{X \subseteq \{1,2,3,\dots,n\}, X \neq \emptyset} (-1)^{|X|-1} \cdot \bigcap_{x \in X} S_x$$

由此式子可以直接发现容斥原理通过计算交的大小来计算并的大小的目的。

而后者则是实战应用中更常见的容斥原理的应用。譬如我们要求不包含 S 中的元素的方案数, 我们肯定要拿总方案数减去包含 S 中元素的方案数, 怎么求后者呢? 我们枚举一个 S 的子集 S' , 然后强制要求 S' 中的元素必须被选, 然后贡献加上方案数乘以 $(-1)^{|S'|}$ 即可。

为什么这样做是正确的? 其实我们容斥原理的目的, 实际上是要让每个不合法的方案数被计算的次数都是 0, 每个合法的方案数被计算的次数都是 1, 这不难让我们想到一个式子 $[n=0] = \sum_{i=0}^n \binom{n}{i} (-1)^i$, 而根据实际意义, 对于一种方案, 其对应的容斥系数就是上式将 n 换为【这种方案选择的子集与 S 的交的大小】后的结果, 其 $= 0$ 表明其符合要求, 其 > 0 表明其不合要求, 也就证明了该算法的正确性。

这里稍微提几个可以用容斥解决的问题:

- 符合某种条件的 DAG 个数, 一般需要子集 DP, 这里容斥的地方在于枚举入度为 0 的点。
- 大多数包含“恰好”字眼的题, 要么用简单的差分容斥解决, 要么使用组合恒等式 $[n=0] =$

$\sum_{i=0}^n \binom{n}{i} (-1)^i$ 设计适当的容斥系数。

- 要求每个元素都不符合某条件的题，容斥哪些元素符合这个条件。

有时候如果贡献很难算，也可以尝试容斥，正难则反，多方面地想想说不定有收获。

下面将详细介绍三个与容斥有很大联系模型。

4.2.1. 二项式反演

所谓反演，就是考虑我们已知用 b 表示 a 的形式，现在我们需要反过来用 b 表示 a ，如果式子简单一点那肯定是容易的，式子复杂了就需要一些技巧，下面两个部分介绍两种和容斥关系非常密切的反演：二项式反演和 Min-Max 反演。

为什么说它们和容斥关系密切呢？因为它们都要用到一个恒等式： $\sum_{i=0}^n (-1)^i \binom{n}{i} = [n=0]$ 。

二项式反演可以用来解决部分计算“恰好 k 个满足 xxx 条件的方案数”的题目。

给定 $a_n = \sum_{i=0}^n b_i \binom{n}{i}$ ，那么有 $b_n = \sum_{i=0}^n (-1)^{n-i} \binom{n}{i} a_i$

证明：

$$\begin{aligned} & \sum_{i=0}^n (-1)^{n-i} \binom{n}{i} a_i \\ &= \sum_{i=0}^n (-1)^{n-i} \binom{n}{i} \sum_{j=0}^i b_j \binom{i}{j} \\ &= \sum_{j=0}^n b_j \sum_{i=j}^n (-1)^{n-i} \binom{n}{i} \binom{i}{j} \\ &= \sum_{j=0}^n b_j \binom{n}{j} \sum_{i=j}^n (-1)^{n-i} \binom{n-j}{i-j} \cdot 1^{i-j} \\ &= \sum_{j=0}^n b_j \binom{n}{j} [j=n] \\ &= b_n \end{aligned}$$

怎么来解决上面的问题呢？设 a_i 表示钦定 i 个满足条件的方案数， b_i 表示恰好 i 个满足条件的方案数，发现 a, b 满足上面第一个等式，那么可以通过反演求出 b 。有的时候需要卷积优化到 $n \log n$ 。

4.2.2. Min-Max 反演

又称 Min-Max 容斥。

$$\text{Max}(S) = \sum_{T \subseteq S, T \neq \emptyset} (-1)^{|T|-1} \text{Min}(T)$$

把 $\min, \max$ 互换等式依然成立。

证明：考察集合中第 k 大的被计算的次数：

- 如果 $k = 1$ 显然为 1。
- 如果 $k = 2$ 则等于 $\sum_{j=0}^{k-1} \binom{k-1}{j} (-1)^j$ ，发现又是上面的经典二项式定理，其等于 $[k-1 = 0] = 0$ 。

因此只有 Max 会算入答案。

Min-Max 反演广泛应用于期望题中，有的题让我们求出所有点都被选中的期望时间，此时我们可以通过 Min-Max 容斥将其变为选定一个子集，求其至少有一个点被选中的期望时间，有时候后者的难度远低于前者，我们就可以采用 Min-Max 容斥。一般难一点的题不会允许暴力枚举 T 复杂度通过，此时你就要挖掘一些性质，比方说将【求其至少有一个点被选中的期望时间】写的简洁一点，然后用类似于 DP 的方法求解，具体情况具体分析，这里不再展开。

事实上 Min-Max 容斥还有一个扩展版本，大概就是容斥系数稍微改一改，系数乘上一个 $\binom{|T|-1}{k-1}$ ，由于不太常用所以这里不再赘述，类似的式子也可以自己推得。

4.2.3. 卡特兰数与折线翻折模型

卡特兰数 C_n 表示从原点到 (n, n) 只能向上走或向右走且不经 $y = x$ 上方的点的方案数。

发现如果一条路径经过 $y = x$ 上方的点，那么其必然穿过 $y = x + 1$ 。因此考虑拿 $\binom{2n}{n}$ 减去穿过 $y = x + 1$ 的方案数，后者可以将终点关于该直线翻折，发现是从 $(0, 0)$ 到 $(n+1, n-1)$ 的方案数，可得 $C_n = \binom{2n}{n} - \binom{2n}{n-1} = \frac{1}{n+1} \binom{2n}{n}$ 。

事实上有很多可以用类似的折线翻折模型去解决的题目。如果只涉及一个折线那直接翻即可，如果涉及到两个折线显然翻一次就不行了，那有什么办法呢？

实际上也是可以做的，详情可见 2021 集训队互测细菌，具体方法就是：

- 先求出原答案。
- 将原点沿 A 翻折，减去对应方案数，再沿 B 翻折，加上对应方案数，再沿 A 翻折，减去对应方案数，再沿 B 翻折，加上对应方案数，以此类推直到不论怎么翻方案数都是 0。
- 将原点归位，先翻 B 做一遍相同的操作。

正确性显然。

碰到类似的题，可以考虑将这个过程放到坐标系上，然后将其转化为折线翻折的题。对于两条折线的折线翻折问题，有一个比较可利用的性质是，如果步长为 k ，那么翻折次数大约是 $\frac{n}{k}$ 级别的，如果 k 很大，可以利用此性质进行根号分治。

4.3. 单位根反演

单位根反演可以解决不少下标模常数等于定值的项对应的组合数求和的问题。具体来说单位根反演的式子如下：

$$\frac{1}{n} \sum_{i=0}^{n-1} \omega_n^{im} = [m \equiv 0 \pmod{N}]$$

证明：显然如果 $m \bmod n = 0$ 那么式子中每一项都是 1，式子的值为 1，否则原式可以化简为 $\frac{1}{n} \cdot \frac{(\omega_n^{mn} - 1)}{\omega_n^m - 1} = 0$ 。

通常式子中带有下标有关的同余符号，并且题目保证了一些与 $\text{MOD} - 1$ 有关的奇怪整除条件时都可以考虑单位根反演。

4.4. 形式幂级数

所谓形式幂级数，就是一类和多项式形式相同的式子，即 $f(x) = \sum_{i=0}^n a_i x^i$ ，但是我们不考虑具体对于某个 x ， $f(x)$ 的取值，而只关心它前面的系数的式子。同时这个式子又满足一般的多项式的四则运算规律。在很多情况下我们可以通过对形式幂级数进行各种各样的变换，通过变换后的系数来进行计数。

一些记号： $[x^n]f(x)$ 表示 $f(x)$ 中 x^n 前的系数。

4.4.1. 多项式乘法

考虑两个多项式 $F(x), G(x)$ ，它们的系数分别为 $f_0 \sim f_n, g_0 \sim g_m$ ，它们相乘得到的结果为 $H(x)$ ，系数为 $h_0 \sim h_{n+m}$ ，那么显然有

$$h_i = \sum_{x+y=i} f_x g_y$$

我们考虑令 N 为 $> n+m$ 的最小的 2 的幂，那么有

$$h_i = \sum_{x+y \equiv i \pmod{N}} f_x g_y$$

调用单位根反演可以有

$$h_i = \frac{1}{N} \sum_{x,y} f_x g_y \sum_{j=0}^{N-1} \omega_N^{(x+y-i)j}$$

$$h_i = \frac{1}{N} \sum_{j=0}^{N-1} \omega_N^{-ij} \sum_x f_x \omega_N^{xj} \cdot \sum_y g_y \omega_N^{yj}$$

如果我们记 $\hat{F}(x)$ 满足 $[x^i]\hat{F}(x) = \sum_{j=0}^n f_j \omega_N^{ij}$ ， $\hat{G}(x)$ 也同理，那么 $h_i = \frac{1}{N} \sum_{j=0}^{N-1} \omega_N^{-ij} \hat{f}_j \hat{g}_j$ 。

可以注意到后面的式子和前面的形式类似，只不过 ω 的指数上带个负号，最后乘个 $\frac{1}{N}$ 罢了，因此我们只考虑 $f \rightarrow \hat{f}$ 的变换怎么处理，这也就是 DFT 的过程。考虑将下标为项和偶数的项提取出来，组成两个长度 $\frac{N}{2}$ 的数组 f_0, f_1 ，先递归求出 $\hat{f}_0, \hat{f}_1$ ，考虑怎么合并，由于 $\omega_N^{2j} = \omega_{N/2}^j$ ， $\omega_N^{j+N/2} = -\omega_N^j$ ，可以得到：

$$\hat{f}_i = \sum_{j \equiv 0 \pmod{2}} f_j \omega_N^{ij} + \sum_{j \equiv 1 \pmod{2}} f_j \omega_N^{ij}$$

$$\hat{f}_i = \sum_{j \equiv 0 \pmod{2}} f_j (\omega_{N/2}^{j/2})^i + \sum_{j \equiv 1 \pmod{2}} f_j (\omega_{N/2}^{(j-1)/2})^i \cdot \omega_N^i$$

$$\hat{f}_i = \begin{cases} \hat{f}_0^i + \omega_N^i \cdot \hat{f}_1^i & (i < \frac{N}{2}) \\ \hat{f}_0^{i-N/2} - \omega_N^{i-N/2} \cdot \hat{f}_1^{i-N/2} & (i \geq \frac{N}{2}) \end{cases}$$

分治处理即可。

直接递归一个数组返回一个数组常数太大了，我们考虑先将其分成相应的组，这样每次相当于只是递归左半边和右半边然后合并。发现最后分组得到的结果中， a_i 会跑到 $rev(i)$ 的位置上，其中 $rev(i)$ 表示将 i 补成 $\log_2(N)$ 位二进制数后从头到尾 reverse 后得到的二进制数，可以简单递推求出，这样可以小常数实现。

FFT 和 NTT 是两种处理运算中涉及到的复数的方式。FFT 是直接复数类存复数，NTT 则是将运算放到模 M 的剩余系。但是使用 NTT 对 M 有一些要求，具体来说 ω_N^i 有定义的条件如下：

- M 存在原根。
- $\varphi(M)$ 是 N 的倍数。由于 N 是 2 的幂，这条规则即要求 $\varphi(M)$ 中 2 的次数 $\geq \log_2(N)$ 。

而一般题目中的 M 是质数。由此看来，一般情况下只有当模数是形如 $A \cdot 2^k + 1$ 型的质数时才可以进行 NTT，998244353 和 1004535809 是常用模数。

以上通过两个序列生成第三个序列的方式被称为卷积。对于这种两数乘积贡献到它们下标之和的卷积，被称为加法卷积，卷积种类有很多种，FFT/NTT 还能解决另一种减法卷积，即 $h_i = \sum_{x-y=i} f_x g_y$ ，具体方法就是把一个多项式系数 reverse 一下，这样两下标之差为定值可以变为两下标之和为定值。

4.4.2. 多项式基础操作

学会了多项式乘法，我们就可以由此进行多项式的各种操作，包括但不限于求逆、开根、求 $\ln$ 、求 $\exp$ 、多项式快速幂，其主要思想都是倍增，即根据待求多项式 $F(x) \bmod x^{n/2}$ 一步步求出 $F(x) \bmod x^n$ 。这里将复习以上提到的四个知识点，以及一些多项式解题的常用策略。

4.4.2.1. 多项式 inv。 对于多项式 $F(x)$ 求出满足 $F(x)G(x) \equiv 1 \pmod{x^n}$ 的 $G(x)$ 。

在下文中，我们均假设 n 是二的整数次幂，如果不是则在开头自动补 0 即可达到同样的效果。

我们递归求出 $G(x) \bmod x^{n/2}$ 的值 $G_0(x)$ ，那么显然

$$G(x) - G_0(x) \equiv 0 \pmod{x^{n/2}}$$

$$(G(x) - G_0(x))^2 \equiv 0 \pmod{x^n}$$

$$G^2(x) + G_0^2(x) - 2G_x G_0(x) \equiv 0 \pmod{x^n}$$

$$G(x) + G_0^2(x)F(x) - 2G_0(x) \equiv 0 \pmod{x^n}$$

$$G(x) \equiv 2G_0(x) - G_0^2(x)F(x) \pmod{n}$$

4.4.2.2. 多项式 sqrt. 对于多项式 $F(x)$ 求出满足 $G^2(x) \equiv F(x) \pmod{x^n}$ 的 $G(x)$ 。

我们假设 $F(x)$ 常数项为 1, 否则则需要通过二次剩余求出 $G(x)$ 常数项, 这里不再赘述。

还是假设求出 $G_0(x)$, 那么有

$$(G(x) - G_0(x))^2 \equiv 0 \pmod{x^n}$$

$$G^2(x) - 2G(x)G_0(x) + G_0^2(x) \equiv 0 \pmod{x^n}$$

$$F(x) - 2G(x)G_0(x) + G_0^2(x) \equiv 0 \pmod{x^n}$$

$$G(x) \equiv \frac{F(x) + G_0^2(x)}{2G_0(x)} \pmod{x^n}$$

根据主定理时间复杂度 $T(n) = T(\frac{n}{2}) + n \log n = n \log n$ 。

4.4.2.3. 多项式 ln. 给定多项式 $F(x)$, 求 $G(x) = \ln F(x)$, 即 $\exp G(x) = F(x)$ 的 G 。

至于这个 $\log$ 和 $\exp$ 是怎么定义在形式幂级数上的, 一种理解是求导, 即 $G(x)$ 满足 $G'(x) = \frac{F'(x)}{F(x)}$, 显然这样可以一项一项推出 $G(x)$ 。还有一种理解是用 $\exp$ 的麦克劳林展开式, $e^x = \sum_{i=0}^{\infty} \frac{x^i}{i!}$, 这样我们也可以得到 $F(x) = \sum_{i=0}^{\infty} \frac{G(x)^i}{i!}$, 如果 $F(x)$ 的常数项为 1, 那么必然存在一个常数项为 0 的多项式 $G(x)$ 满足该条件。显然对于一个有限的 n , 上面的展开式中对于 $i > n$, $\frac{G(x)^i}{i!}$ 中第 n 项就以及为 0 了, 所以我们通过 $F(x)$ 第 n 项的值以及 $G(x)$ 的前 $n-1$ 项推出第 n 项。

然后就是快速求的问题了, 其实直接根据第一种理解就可以得出来, $G'(x) = \frac{F'(x)}{F(x)}$, 求出 $G'(x)$ 积分回去即可。

4.4.2.4. 多项式 exp. 给定多项式 $F(x)$, 求 $G(x) = \exp F(x)$ 。

还是假设我们已经求出了 $G(x) \bmod x^{n/2} = G_0(x)$, 考虑函数 $A(G(x)) = \ln G(x) - F(x)$, 那么显然我们所求即为 $A(G(x))$ 零点, 在 $G_0(x)$ 处对 $A(G(x))$ 进行泰勒展开可以得到:

$$A(G(x)) \equiv A(G_0(x)) + (G(x) - G_0(x)) \cdot A'(G_0(x)) + (G(x) - G_0(x))^2 \cdot A''(G_0(x)) + \dots \pmod{x^n}$$

不过我们发现从第二项开始, 由于 $(G(x) - G_0(x))^2$ 的存在, 其 $\bmod x^n$ 的值已经为零了, 因此我们只需求 $A(G_0(x)) + (G(x) - G_0(x)) \cdot A'(G_0(x))$ 的零点。即

$$\ln G_0(x) - F(x) + (G(x) - G_0(x)) \cdot \frac{1}{G_0(x)} \equiv 0 \pmod{x^n}$$

$$\ln G_0(x) - F(x) - 1 + \frac{G(x)}{G_0(x)} \equiv 0 \pmod{x^n}$$

$$G(x) \equiv (1 + F(x) - \ln G_0(x)) \cdot G_0(x) \pmod{x^n}$$

根据主定理, 复杂度还是 $1\log$ 。

4.4.2.5. 多项式快速幂。 给定多项式 $F(x)$ 和 k , 要求 $G(x) = F(x)^k$ 。

显然可以 $n \log n \log k$, 这里不再赘述。

如果 $F(x)$ 常数项为 1, 那么显然 $\ln G(x) = k \ln F(x)$ 。可以 $\ln$ 一下然后 $\exp$ 回去, 这样复杂度和 k 无关。

4.4.2.6. 分治 NTT。 所谓分治 NTT, 就是分治内部 NTT, 一般可以用来解决以下两种问题:

- 给定序列 f , 要求序列 g 满足 $g_0 = 1, g_i = \sum_{j=1}^i g_{i-j} f_j$
- 求 $\prod_{i=1}^n F_i(x)$, 其中 $F_i(x)$ 长度总和是 $O(n)$ 的。

对于第一种, 做法就是 cdq 分治, 分治 $[l, r]$ 时, 先分治左边算出左边的 g , 然后用 NTT 算出左 $\rightarrow$ 右的贡献。然后再分治 NTT 右边。时间复杂度是 $2\log$ 的, 有时候这种形式的分治 NTT 也可以用多项式求逆代替, 这样复杂度会少一个 $\log$ 。

对于第二章, 做法就是分治求出左边的 $\prod$ 和右边的 $\prod$, 合并时候启发式合并起来即可, 根据启发式思想复杂度还是 $2\log$ 的。

4.4.2.7. 倍增 NTT。 倍增 NTT 一般可以优化一类满足通过 $dp_{x,*}$ 和 $dp_{y,*}$, 可以通过卷积唯一确定 $dp_{x+y,*}$ 的 DP。这种题第一维一般会很大, 是 114514 个 $k \leq 10^{18}$ 型模型之一。

具体思路就是先倍增求出所有 $dp_{2^k,x}$, 然后一路合并得到 $dp_{x,*}$ 即可。时间复杂度 $n \log k \log n$ 。

4.4.3. 生成函数

4.4.3.1. OGF 与 EGF。 对于一个序列 a (长度可以有限也可以无限), 定义其 OGF 为 $A(x) = \sum_{n \geq 0} x^n a_n$, 也就是直接以 a 为系数构造多项式, 这样两个序列 a, b 进行加法卷积后得到的序列 c 的 OGF 就是 $A(x)$ 与 $B(x)$ 进行乘法后的结果。

OGF 用于解决无标号问题, 因为无标号问题各部分的贡献乘起来时不需要乘任何其他系数。

常用无限序列的 OGF 的封闭形式 (即不带无穷级数的形式):

- $\{1, 1, 1, \dots\}$: $\frac{1}{1-x}$

- $\{1, 2, 3, \dots\} : \frac{1}{(1-x)^2}$
- $\left\{\binom{k-1}{k-1}, \binom{k}{k-1}, \binom{k+1}{k-1}, \binom{k+2}{k-1}, \dots\right\} : \frac{1}{(1-x)^k}$
- $\{0, 1, \frac{1}{2}, \frac{1}{3}, \dots\} : -\ln(1-x)$

与 OGF 相对的是 EGF。定义一个序列的 EGF 为 $A(x) = \sum_{n \geq 0} a_n \cdot \frac{x^n}{n!}$ 。类比 OGF, EGF 的性质是 a, b 的 EGF 进行多项式乘法后会得到 a, b 进行二项加法卷积后的 EGF。其中两个序列 a, b 的二项加法卷积为 $c_n = \sum_{x+y=n} \binom{n}{x} a_x b_y$ 。

EGF 用于解决有标号问题, 有标号问题将各个部分合并起来的时候实际上是在做关于点数的二项加法卷积。

下面也给出了几个 EGF 的封闭形式:

- $\{1, 1, 1, \dots\} : e^x$ 。
- $\{1, c, c^2, c^3, \dots\} : e^{cx}$
- $\{1, 0, 1, 0, \dots\} : \frac{e^x + e^{-x}}{2}$

4.4.3.2. exp 的实际意义。 事实上, 关于 EGF 的 exp, 还有一个更深刻的实际意义: 集合的集合。

具体来说, 假设 a_i 表示有 i 个点的集合个数的 EGF, $A(x)$ 为其 EGF。那么 $\exp A(x)$ 为 b_i 的 EGF, 其中 b_i 表示由集合的集合组成的, 含有 i 个点的本质不同的集合个数, 值得注意的是, 集合与集合之间不考虑顺序, 但集合内部的点存在顺序, 也就是集合中的点是有标号的, 但集合是不做区分的。

举几个简单的例子:

- 如果连通图是上文中的“集合”, 那么图就是“集合的集合”, 知道了 n 个点有标号图的 EGF, $\exp$ 一下就是 n 个点图的 EGF, 反过来则需进行 $\ln$ 操作。
- 如果树是集合, 那么森林就是集合的集合, 二者之间同样存在 $\ln / \exp$ 的关系。
- 如果欧拉连通图是集合, 那么每个点度都是偶数的图就是集合的集合, 二者之间同样存在 $\ln / \exp$ 的关系。

知道道理的话很简单, 因为 $\exp A(x) = \sum_{n \geq 0} \frac{A(x)^n}{n!}$, 相当于枚举集合的集合中有多少个集合, EGF 乘起来, 由于集合之间不做区分, 相应地除掉 $n!$ 。

4.4.3.3. Prufer 序列。 Prufer 序列是帮助计算一些特定要求的树的有力工具之一, 同时也是生成等概率随机树的必需品。由于其与组合数学关系更密切, 就放到这里复习了。

大概就是, 对于一棵无根树, 考虑以 n 为根, 每次删去编号最小的叶子并记录下与其相连的节点, 如此操作直到只剩下两个点, 这样可以得到长度 $n-2$ 的值域 $[1, n]$ 的序列, 可以证明一棵无根树与一个这样的序列构成双射。

显然一棵树可以对应一个序列。而对于一个序列, 显然每个点的度等于其在序列中出

现次数加一，由此我们也可以模拟出这棵树，并且对于任意序列我们可以记录出每个数（最后一次出现位置, 数的大小）的二元组，那么每个点的父亲对应的二元组肯定比这个点的二元组小，所以连出来的必然是一棵树。

由此我们可以得出结论： n 个点组成的无根树个数为 n^{n-2} 。

此外该定理还有一个扩展：假设现在树中有 k 个连通块，大小分别为 $a_1, a_2, \dots, a_k$ ，把它们连成一棵树的方案数为 $\prod_{i=1}^k a_i \cdot n^{k-2}$ 。

4.4.4 拉格朗日插值

给定平面上 $n+1$ 个 x 互不相同的点 (x_i, y_i) ，求一个多项式同时经过这些点。

可以证明这个多项式为 $\sum_{i=1}^{n+1} y_i \cdot \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$ ，证明可以依次带入点值 $x_1, x_2, \dots, x_{n+1}$ 检验。

如果要求该多项式在某 $x = k$ 处的点值，那么暴力显然是 n^2 的，如果 $x_1 \sim x_{n+1}$ 成一个连续的区间，那么插值可以做到线性，具体方法就是记录 $k - x_i$ 的前后缀积，以及阶乘的逆元，这样可以 $O(1)$ 计算后面的 $\prod$ 。

如果要求该多项式的系数，那么如果不使用高端技巧只能做到平方，方法就是分子上的多项式你先预处理 $\prod (x - x_i)$ ，然后每次枚举一个 (x_i, y_i) 就把 $x - x_i$ 的部分除掉，分母显然暴力枚举复杂度没有问题。

4.5. 集合幂级数

4.5.1. FWT

所谓“幂级数”，就是一个形如若干个形式变量的幂次相乘，然后相加，前面有一些对应的系数的多项式。类比形式幂级数，我们定义集合幂级数为 $F(x) = \sum_S a_S x^S$ ，即每个下标都是一个集合（二进制数），我们还是用符合 $[x^S]F(x)$ 表示 x^S 前的系数。

定义两个集合幂级数 $F(x), G(x)$ 的 OR（并）、AND（交）、XOR（对称差）卷积，分别表示 $[x^S]F(x) \cdot [x^T]G(x)$ 会贡献到 $x^{S|T}, x^{S \& T}, x^{S \oplus T}$ 处的卷积，用序列的角度就是给定两序列 a, b ，那么它们的 OR、AND、XOR 卷积分别表示满足以下条件的序列 c ：

- OR: $c_i = \sum_{x|y=i} a_x b_y$
- AND: $c_i = \sum_{x \& y=i} a_x b_y$
- XOR: $c_i = \sum_{x \oplus y=i} a_x b_y$

类比 FFT，我们希望构造适当的变换将其转化为点值相乘的形式，然后再逆变换将点值变回原来的序列。如何构造呢？我们依次来看下三种卷积。

OR 卷积

我们定义 $\hat{a}_i = \sum_{j|i} a_j$ ，那么

$$\begin{aligned}
\hat{c}_i &= \sum_{j|i=i} c_j \\
&= \sum_{j|i=i} \sum_{x|y=j} \left(\sum_{p|x=x} a_p \right) \cdot \left(\sum_{q|y=y} b_q \right) \\
&= \sum_{p|i=i} a_p \cdot \sum_{q|i=i} b_q \\
&= \hat{a}_i \cdot \hat{b}_i
\end{aligned}$$

接下来考虑如何求 $\hat{a}_j$ ，考虑分治，我们假设要对长度为 2^k 的序列 a 实现 $a \rightarrow \hat{a}$ ，那么我们就先将 a 划分为前半段和后半段，记前半段记作 $a0$ ，满足 $a0_i = a_i$ ，后半段记作 $a1$ ，满足 $a1_i = a_{i+2^{k-1}}$ 。那么我们先递归求出 $\hat{a}0$ 和 $\hat{a}1$ ，考虑对于一个 $0 \leq i < 2^{k-1}$ ，如何通过 $\hat{a}0_i$ 和 $\hat{a}1_i$ 求出 $\hat{a}_i$ 和 $\hat{a}_{2^{k-1}+i}$ ，由于 i 的 2^{k-1} 位为 0，所以 $\hat{a}1_i$ 不会对其有贡献， $\hat{a}_i = \hat{a}0_i$ ，而 $i + 2^{k-1}$ 的 2^{k-1} 位为 1，其会同时接受 $\hat{a}0_i$ 和 $\hat{a}1_i$ 的贡献，故 $\hat{a}_{i+2^{k-1}} = \hat{a}1_i + \hat{a}0_i$ 。时间复杂度 $n \log n$ 。

AND 卷积

与 AND 卷积类似，只不过我们的定义改为 $\hat{a}_i = \sum_{j \& i = i} a_j$ ，最后将左右两部分的 $\hat{a}$ 值合并起来的部分改为 $\hat{a}_{i+2^{k-1}} = \hat{a}1_i$ ， $\hat{a}_i = \hat{a}1_i + \hat{a}0_i$ 。时间复杂度同上。

XOR 卷积

由于对称差这个运算与集合取并和取交有着本质上的区别，所以 XOR 卷积的处理方法与前者有较大区别。

对于 XOR 卷积，我们定义

$$\hat{a}_i = \sum_{j=0}^{2^k-1} (-1)^{\text{popcount}(i \oplus j)} \cdot a_j$$

那么对应地有

$$\begin{aligned}
\hat{c}_i &= \sum_{j=0}^{2^k-1} (-1)^{\text{popcount}(i \& j)} \cdot c_j \\
&= \sum_{j=0}^{2^k-1} (-1)^{\text{popcount}(i \& j)} \cdot \sum_{x \oplus y = j} a_x b_y \\
&= \sum_{j=0}^{2^k-1} \sum_{x \oplus y = j} a_x b_y \cdot (-1)^{\text{popcount}(x \& i)} \cdot (-1)^{\text{popcount}(y \& i)} \\
&= \sum_x a_x \cdot (-1)^{\text{popcount}(x \& i)} \cdot \sum_y b_y \cdot (-1)^{\text{popcount}(y \& i)} \\
&= \hat{a}_i \cdot \hat{b}_i
\end{aligned}$$

还是考虑如何实现 $a \rightarrow \hat{a}$ ，还是考虑分治，我们假设要对长度为 2^k 的序列 a 实现 $a \rightarrow \hat{a}$ ，那么我们还是先将 a 划分为前半段 $a0$ 和后半段 $a1$ ，考虑 $\hat{a}0_i, \hat{a}1_i$ 对 $\hat{a}_i$ 和 $\hat{a}_{i+2^{k-1}}$ ，由于只有当 i, j 某一位都是 1 的时候， $(-1)^{\text{popcount}(i \& j)}$ 的符号才会改变，因此 $\hat{a}0_i \rightarrow \hat{a}_i, \hat{a}0_i \rightarrow \hat{a}_{i+2^{k-1}}, \hat{a}1_i \rightarrow \hat{a}_i$ 的系数都是 1， $\hat{a}1_i \rightarrow \hat{a}_{i+2^{k-1}}$ 的贡献为 -1 ，即 $\hat{a}_i = \hat{a}0_i + \hat{a}1_i, \hat{a}_{i+2^{k-1}} = \hat{a}0_i - \hat{a}1_i$ 。值得注意的是，该变换的逆变换，即 $\hat{a} \rightarrow a$ ，可以视作对 $\hat{a}$ 进行正向变换，然后每项都乘以 $\frac{1}{2^k}$ 。

以上三个卷积复杂度均为 $n \log n$ 。

FWT 变换的一个很基本的特点是它们都是线性变换，即我们可以用一个矩阵来刻画这个变换，因为对于任意一种运算，都有 $a\text{FWT}(p) + b\text{FWT}(q) = \text{FWT}(ap + bq)$ 。

4.5.2. k 进制 FWT

在上面的流程中，针对下标的运算都是二进制运算，我们尝试将其扩展到 k 进制下，这样取 OR、取 AND、取 XOR 分别对应每一位取 $\max, \min$ 和模 k 意义下的加法。

OR 和 AND 同样是容易的，每次分治的时候，将原序列平均分成 k 个序列 $b_{i,j} (0 \leq i < k, 0 \leq j < \frac{n}{k})$ ，然后对于 OR 卷积，就直接令 $\hat{a}_{i, \frac{n}{k}+j} = \sum_{c=0}^i \hat{b}_{c,j}$ ，对于 AND 卷积，令 $\hat{a}_{i, \frac{n}{k}+j} = \sum_{c=i}^{k-1} \hat{b}_{c,j}$ 即可。

比较难的地方在于 XOR 卷积，不难发现上面的变换之所以能将卷积变为点乘，主要是因为对于任意 $x \oplus y = j$ 以及 i ，都有 $(-1)^{\text{popcount}(x \& i)} \cdot (-1)^{\text{popcount}(y \& i)} = (-1)^{\text{popcount}(j \& i)}$ 。思考在 k 进制下什么变换函数 f 满足这个条件，这等价于对于任意 $x, y, i < k$ ，都有 $f(x, i) \cdot f(y, i) = f((x + y) \bmod k, i)$ 。考虑 k 次单位根 ω ，容易验证 $f(x, y) = \omega_k^{xy}$ 满足条件。于是对于 XOR 卷积，令 $\hat{a}_{i, \frac{n}{k}+j} = \sum_{c=0}^{k-1} \omega_k^{ci} \hat{b}_{c,j}$ 即可满足条件。

如果 $\varphi(p)$ 是 k 的倍数，那么在 $\bmod p$ 意义下做 k 进制 FWT 可以做到 $n \log_k n \cdot k^2$ ，否则需要扩域，由于扩域后乘法是 k^2 的，所以复杂度 $n \log_k n \cdot k^4$ 。一般 k 不会太大。

4.5.3. 子集卷积

给定序列 a, b ，定义其子集卷积为

$$c_i = \sum_{x|y=i, x \& y=0} a_x b_y$$

直接用 FWTor/FWTand 的角度设计变换函数似乎不太可做。不过我们发现一个性质，就是对于任意 x, y , $\text{popcount}(x) + \text{popcount}(y) \geq \text{popcount}(i)$, 且等号取到当且仅当 $x \& y = 0$, 因此我们考虑将一维数组扩展成二维, 初始 $a_{\text{popcount}(i), i} = a_i, b_{\text{popcount}(i), i} = b_i$, 然后对于所有 $i \in [0, \log_2(n)]$, 对 a_i, b_i 进行 FWTor。然后再对于 $i \in [0, n-1]$, 把 $a_{j,i}$ 和 $b_{j,i}$ 作卷积得到 c 数组, 每一位逆变换回去后 $c_{\text{popcount}(i), i}$ 就是我们想要的 c_i 。

4.6. 线性代数

4.6.1. 高斯消元

高斯消元最基础的应用是求解形如

$$\begin{cases} a_{1,1}x_1 + a_{1,2}x_2 + \cdots + a_{1,n}x_n = a_{1,n+1} \\ a_{2,1}x_1 + a_{2,2}x_2 + \cdots + a_{2,n}x_n = a_{2,n+1} \\ \vdots \\ a_{n,1}x_1 + a_{n,2}x_2 + \cdots + a_{n,n}x_n = a_{n,n+1} \end{cases}$$

具体方法就是, 先找一个不为 0 的 $a_{i,1}$ 作为第 1 行, 如果找不到这样的行则说明该方程组不相容 (即要么不存在解, 要么存在无穷多解), 然后对于第 j 行 ($2 \leq j \leq n$), 将第 1 行的 $-\frac{a_{j,1}}{a_{1,1}}$ 倍加到第 i 行上, 这样可以消去第 j 个方程 x_1 前的系数, 然后对第 $2 \sim n-1$ 行的后 $n-1$ 列递归进行此操作即可。这样矩阵被削成了这个样子

$$\begin{bmatrix} * & * & * & * & \cdots & * & * \\ 0 & * & * & * & \cdots & * & * \\ 0 & 0 & * & * & \cdots & * & * \\ 0 & 0 & 0 & * & \cdots & * & * \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & * & * \end{bmatrix}$$

注意到最后一行只有 x_n 前的系数非零, 这样可以解出 x_n , 将 x_n 带入第 $n-1$ 个方程即可解出 x_{n-1} , 以此类推直到解出 x_1 即可。

高斯消元也可以扩展到方程数不等于未知变量数的情况, 这样高斯消元可以在 $O(nm \min(n, m))$ 的复杂度内完成, 方法和之前类似, 这里不再展开。

注意: 如果要对 q 个矩阵高斯消元, 但是这些矩阵除了最后一列 (也就是等号右边的部分) 不同之外, 其他部分都相同, 则可将这些矩阵的常数列并到一起在 $n^2(n+q)$ 的复杂度内进行高斯消元, 即常数列编号 $n+1 \sim n+q$, 不必 qn^3 地进行。

4.6.2. 线性基

给定 n 个二进制数 $a_1 \sim a_n$, 要求由这些数的某个子集组成的异或和有哪些。

如果把每个二进制位看成向量的一维, 那么两个向量的异或和可以视作其在 $\text{mod } 2$ 意义下的加法, 于是该问题实际上等价于求一个向量集合能够生成的空间, 考虑使用高斯消元。

但由于值域只有 $0/1$, 可以考虑采用不同寻常的方式进行高斯消元, 具体来说我们记 b_i 表示最高位为 2^i 的向量是什么, 插入一个二进制数时, 我们从高到低考虑这些位, 如果对应位为 1, 那么查看当前位的 b_i , 如果为 0 则令 b_i 为当前的数值, 否则令当前的数值变为其与 b_i 的异或, 容易证明其与高斯消元等价。这样所有非零的 b_i 就组成了原序列的一组基, 由于每个 b_i 最多被插入一次, 所以基的大小 $\leq \log n$ 。

线性基可以用来做以下事情:

- 查询最大异或子集, 具体方法就是按位贪心, 从高位到低位贪心, 如果当前数异或 b_i 则异或, 否则什么都不干。
- 查询一个数是否能表示成某个子集的异或和, 具体方法就是重复一遍插入的过程, 如果能成功插入则表明不能。

如果要求一个子集能够异或出来的数中第 k 大的值, 那么应当在插入 b_i 时做以下操作:

- 从大到小遍历所有 $j < i$, 如果 b_i 第 j 位为 1, 那么令 $b_i \leftarrow b_i \oplus b_j$ 。
- 从小到大遍历所有 $j > i$, 如果 b_j 第 i 位为 1, 那么令 $b_j \leftarrow b_j \oplus b_i$ 。

容易发现这样可以将原矩阵削成简化阶梯型, 即对于每个 $b_i \neq 0$, 只有 b_i 满足第 i 位为 1。我们假设 $b_i \neq 0$ 的 i 从小到大分别为 $i_0, i_1, \dots, i_{k-1}$, 那么容易证明原序列的所有子集 (可以为空) 共可以表示出来 2^k 种异或和。而其中第 x (0 -indexed) 小的异或和就是所有满足 x 的 2^d 位为 1 的 d 的 b_{i_d} 的异或和。这样插入时间复杂度变成了 $\log^2 n$, 总复杂度 $n \log n + \log^3 n$ 。

值得注意的是, 线性基是支持合并的, 合并复杂度 $\log^2 n$, 因此我们可以使用线段树维护区间线性基。但是线性基不支持删除, 所以如果带删除元素可能需要线段树分治等结构。

4.6.3. 行列式

对于 $n \times n$ 的方阵 A , 定义其行列式为

$$\sum_p (-1)^{\tau(p)} \prod_{i=1}^n A_{i,p_i}$$

也就是枚举每个 $1 \sim n$ 的排列 p , 计算 A_{i,p_i} 的乘积, 乘上 -1 的 p 的逆序对数次方再求和。

考虑怎样在多项式复杂度内求行列式, 考虑行列式的性质:

1. 上三角矩阵的行列式等于其对角线元素的乘积 (显然)

2. 将矩阵进行转置, 行列式不变。证明就, 容易证明对于 $(-1)^{\tau(p)} \prod_{i=1}^n A_{i,p_i} =$

$\sum_{p^{-1}} (-1)^{\tau(p^{-1})} \prod_{i=1}^n A_{i,p_i}^T$, 其中 p^{-1} 为 p 的逆排列, 因此构造 $p \rightarrow p^{-1}$ 的映射即可, 容易证明这是双射。

3. 交换矩阵的两行，行列式变号，这等价于证明交换排列的两个元素后，逆序对奇偶性发生改变，这是容易的。
4. 矩阵中如果出现两行线性相关，那么行列式为 0。证明就，假设这两行为 x, y ，那么对于一个排列而言，交换 p_x, p_y 后， $\prod_{i=1}^n A_{i, p_i}$ 不变，但是逆序对个数奇偶性发生变化，它们的贡献相抵消了。
5. 将一行倍加到另一行上，行列式大小不变，证明大概就把行列式的公式展开，然后发现新矩阵行列式可以写成两个矩阵行列式之和，其中一个是原矩阵，还有一个矩阵存在两行线性相关，因此新矩阵行列式等于原矩阵行列式。

有了这些性质以后我们就可以在 poly 的时间内求解行列式了，具体方法就是，按照高斯消元的过程将矩阵削成上三角矩阵，不难发现这期间我们只会进行两个操作，一是交换两行，二是将一行倍加到另一行上，我们维护一个符号变量 $\text{sgn} = 1$ ，每次进行操作一就将其变为相反数，最后的行列式就是 $\text{sgn} \cdot \prod_{i=1}^n A_{i, i}$ 。

如果模数不是质数，那么不难发现麻烦的地方在于我们要对于两行 x, y ，通过将第 x 行倍加到第 y 行上来使得 $A_{y, 1} = 0$ 。事实上这个操作可以类比辗转相除求 gcd ，具体来说如果 $A_{x, 1} > A_{y, 1}$ 则交换两行，否则将第 x 行的 $-\lfloor \frac{A_{x, 1}}{A_{y, 1}} \rfloor$ 倍加到第 y 行上。这样乍一看是三方 $\log$ 的，容易用势能分析到三方。

行列式的一个很经典的玩法是，由于交换 p 中的两个元素， p 中逆序对个数奇偶性改变，因此如果我们尝试用行列式的组合意义来进行计数，并且我们发现有些我们不想计入答案的东西满足交换 p 中两个元素后答案不变，我们就可以用行列式来使这些不会计入答案的贡献两两抵消，从而只保留我们想要计入答案的部分。

4.6.4. 矩阵求逆

对于一个矩阵 A ，定义其逆矩阵为满足 $AA^{-1} = I$ 的 A^{-1} 。

具体求法：在 A 的右边拼上一个 I 形成一个 $n \times 2n$ 的矩阵，然后通过高斯消元将左半边变成 I ，此时右半边的值就是 A^{-1} 。证明略去。

4.6.5. 矩阵树定理

矩阵树定理可以用来求一张图的生成树个数。具体方法，记 D 为图的度数矩阵，即 $D_{i, i} = \text{deg}_i$ ，其余元素为 0， A 为图的转移矩阵，即 $A_{u, v} = A_{v, u} = u, v$ 之间边的条数，那么这张图的生成树个数就是 $D - A$ 去掉任何一行和一系列后得到的矩阵的行列式。

矩阵树定理有以下推论：

- 对于带权无向图而言，求其所有生成树边权乘积之和，可以将 (u, v, w) 看作 w 条 (u, v) 之间的边，这样可以视作不带权的问题。
- 对于有向图而言，记 D_{in} 表示图的入度矩阵， A 表示图的转移矩阵，那么 $D_{in} - A$ 去掉第 r 行 r 列后的矩阵的行列式则为以 r 为根的外向树个数。同理 $D_{out} - A$ 去掉第 r 行 r 列后矩阵的行列式为以 r 为根的内向树个数。
- 对于带权无向图而言，求其所有生成树边权之和的和，可以将每条边权值看作一个多项式 $1 +$

wx ，这样相当于求所有生成树边权之积之和。由于一次多项式可以进行加减乘除，所以一次多项式的矩阵也可以求行列式。

4.6.6. BEST 定理

给定一个有向图，BEST 定理可以用来计算其欧拉回路个数。

具体方法就是，求出以 1 为根的内向树个数，然后对于每个点而言，将其不在内向树上的边（显然对于根节点而言，有 $\deg_1$ 个不在内向树上的边，对于其他点而言，有 $\deg_i - 1$ 个不在内向树上的边）钦定一个顺序，不难这样有 $A \cdot \deg_1 \cdot \prod_{i=1}^n (\deg_i - 1)!$ 安排方法。可以证明，一种安排方法唯一对应一个欧拉回路，具体方法就是，每次访问一个节点时，如果它还存在不在内向树上的边没有被访问，就访问对应的边，否则就沿着内向树的边向上走，可以证明这样一定会访问所有边，具体证明可以看我的学习笔记，这里不再赘述。

4.6.7. LGV 引理

虽说去年考过了今年考的可能性不大，这里还是写一下。

LGV 引理用于计算特殊图的不相交路径方案数。

大概就是，给定 DAG 上 n 个起点和 n 个终点，我们定义 $A_{i,j}$ 表示从第 i 个起点走到第 j 个终点的方案数，那么考察 $|A|$ 的实际意义，对于一般 DAG 而言，其含义为，考虑每种可能的排列 p ，计算 $s_i \rightarrow t_{p_i}$ 的方案数，乘起来再乘以 $(-1)^{\tau(p)}$ ，因此。

但是对于一些特殊的图譬如网格图，有一个很重要性质是如果 p 中存在逆序对，那么必然存在两个路径相交，我们找到字典序最小的一对逆序对 (x, y) ，找到它们的交点 Z ，那么相当于原来的路径为 $s_x \rightarrow Z \rightarrow t_{p_x}, s_y \rightarrow Z \rightarrow t_{p_y}$ ，我们将后面一半交换，可以得到 $s_x \rightarrow Z \rightarrow t_{p_y}, s_y \rightarrow Z \rightarrow t_{p_x}$ ，这样相当于交换了 p_x, p_y ，逆序对奇偶性改变， $+1$ 和 -1 使该方案的贡献抵消了。这样我们得出结论：所有存在交点的方案的贡献都会被抵消。这样只剩下不交路径的方案了。

4.7. 群论

一个集合 S 和运算 $\times$ 构成一个群，当且仅当其满足：

- 封闭性，对于任意 $x, y \in S$ ，有 $x \times y \in S$ 。
- 结合律，对于任意 $x, y, z \in S$ ，有 $(x \times y) \times z = x \times (y \times z)$ 。
- 单位元，存在 $e \in S$ ，使得 $\forall x \in S$ 都有 $x \times e = x$ 。
- 逆元，对于任意 $x \in S$ ，均存在 $x^{-1} \in S$ 使得 $x \times x^{-1} = e$ 。

NOI 中用到的群论，一般是基于置换群的 Burnside 引理和 Polya 定理。所谓置换群，就是由置换构成的满足以上四个性质的群。譬如由 n 个形如 $(x, x+1, x+2, \dots, n, 1, 2, 3, \dots, x-1)$ 构成的群就是置换群，因为其满足以上四个定理。

Polya 定理和 Burnside 引理一般用于求解数本质不同等价类的问题，即假设现在有个 n 阶置换组成的群 G 和一个由 n 元组组成的集合 S ，那么对于两个元素 $x, y \in S$ ，如果存在置换

$f \in G$ 使得 $f(x) = y$ ，我们就将 x, y 视作本质相同的元素。我们要统计有多少个本质不同的 n 元组。

轨道-稳定子定理

首先有一个很基础的定理：轨道-稳定子定理。对于任意元素 $k \in S$ ，我们假设 Z_k 为满足 $f \in G$ 且 $f(k) = k$ 的 f 构成的集合， E_k 表示满足 $\exists f \in G, s.t. f(k) = k'$ 的 $k' \in S$ 的 k' 构成的集合，那么我们可以证明， $|Z_k||E_k| = |G|$ 。证明大概就考虑设 $Z_k = \{f_1, f_2, \dots, f_c\}$ ， $E_k = \{g_1, g_2, \dots, g_n\}$ 且 $kh_i = g_i$ ，那么我们试图证明 $f_i h_j$ 两两不同且恰好包含了所有 G 中的元素。首先两两不同是显然的。这里不再赘述。而对于任意一个 $f_0 \in G$ ，假设 $kf_0 = g_i$ ，那么我们考虑 $f_0 \cdot h_i^{-1}$ ，显然，其 $\in Z_k$ ，因此 $|Z_k||E_k| = |G|$ 。

Burnside 引理

Burnside 引理的内容为， S 中本质不同的等价类数量为 $\frac{1}{|G|} \sum_{f \in G} G^f$ ，其中 G^f 表示 S 中有多少个元素 x 满足 $f(x) = x$ 。也就是说我们枚举所有置换 f ，统计 S 中有多少个不动点，求和再除以 $|G|$ 就是 S 中等价类个数。证明是容易的，假设一个等价类大小为 x ，那么显然对于该等价类中的所有 k ，都有 $|E_k| = x$ ，而根据轨道-稳定子定理，该等价类其中所有置换会对上述和式产生 $|Z_k| = \frac{|G|}{|E_k|}$ 的贡献，因此该等价类总共会对合适产生 $|G|$ 的贡献，因此该和式的 $\frac{1}{|G|}$ 倍就是等价类个数。

Polya 定理

在 OI 题目中，群论更常见的考法形如“有一个 n 个元素的序列，你需要为每个元素赋值 $[1, m]$ ，计算本质不同序列个数，经过 xxx 变换得到的本质不同不同构序列个数”，此时置换通常会成一个类似于环的东西。以最经典的，置换群中的所有元素都以 $(x, x+1, x+2, \dots, n, 1, 2, 3, \dots, x-1)$ 出现的置换环为例，我们枚举平移了多少格，设为 d ，那么容易发现不动点个数为 $m^{\gcd(d, n)}$ ，由于 $\gcd(d, n)$ 必然是 n 的约数，因此我们考虑枚举这个 $\gcd$ ，设为 d_0 ，那么有 $\varphi(\frac{n}{d_0})$ 个 d 满足 $\gcd(d, n) = d_0$ ，答案就是 $\frac{1}{n} \cdot \sum_{d_0|n} \varphi(\frac{n}{d_0}) m^{d_0}$ 。

注意，有时上面的 n 的数据范围超过了模数，这时一种常用方法是将模数变为 mod^2 。

5. 数据结构方向

5.1. 五种基础数据结构

五种基础数据结构，指线段树、BIT、可持久化线段树、平衡树、可持久化平衡树。

- 线段树：最基础的数据结构，每个节点表示序列中的一个区间，可以支持单点修改和区间查询等操作，如果可以打标记且标记可合并那也可以区间修改，一般用于解决不涉及下标变换（即区间复制、区间 reverse 等操作）、且区间信息可合并的区间问题。
- BIT：也是最基础的数据结构。每个节点表示区间 $[i - \text{lowbit}(i) + 1, i]$ 。一般用于解决不可减信息的单点修改前缀查询问题，和可减信息的单点修改区间查询问题。
- 可持久化线段树：相当于线段树每次修改都保存一个历史版本，具体方法就是单点修改就对根节点到待修改叶子节点这条路径上新建一个节点维护一个新的标记，区间标记则也需对遍历的节点新建一个节点并进行**标记永久化**（当然如果 pushdown 也可以，不过注意要新开节点而不是直接往它原来两个儿子上推，这样空间常数会大得多）
- 平衡树：线段树能解决的东西，平衡树都能解决，其功能比线段树多了一些涉及到下标变换的操作（如插入、删除和区间 reverse），唯一的缺点是常数比线段树大得多。一般我们实现平衡树都使用 fhq-treap，其最大的优点是好写，比 splay 不知道好写到哪里去了。所以除非迫不得已，否则 NOI 遇到平衡树推荐使用 fhq-treap。
- 可持久化平衡树：类比线段树 $\rightarrow$ 可持久化线段树，每次平衡树操作时都新建一个节点，这样平衡树就变成了可持久化平衡树。可持久化平衡树可以用来解决区间复制问题。

5.2. 线段树的高级用法

5.2.1. 动态开点线段树与标记永久化

众所周知，线段树节点数与下标序列的范围同阶，但是如果我们要维护以值域为下标的线段树，下标的范围可能很大，此时直接暴力建树显然不可取，此时我们就需要动态开点线段树了。此时插入的方法就是将节点编号传入参数中并传引用，这样节点数为 $O(n \log V)$ ，因此部分动态开点线段树需要注意空间问题。

如果动态开点线段树需要支持区间修改，那么直接访问到一个有标记的节点就 pushdown 这样空间复杂度大约要乘个 3，碰到有的题卡不过去。怎么办呢？我们发现，如果我们标记不考虑顺序，即，**不考虑打上这个标记的时间**，譬如区间取 min，那么我们就可以干脆直接把标记打在这个地方，然后查询的时候直接将根到待查询节点的标记全部取 min 并起来。这样空间常数就小多了。

线段树合并常常需要用到标记永久化。

5.2.2. 李超线段树

李超线段树是一种用来维护凸包的数据结构，常用于斜率优化中。具体来说它支持往一个区间中加入一条直线 $y = kx + b$ ，并询问过 $x = k$ 的直线在这个位置处的纵坐标的最大值。

李超线段树的核心思想是线段树上每个节点维护这个区间的最优直线，换句话说，这个区间内最有可能成为最优答案的直线。这个定义看上去有点别扭，事实上我们希望对于任意一个节点而言，经过这个位置的直线中在这个点处的 y 最大的直线都是这个点到根路径上某个点的“最优直线”。考虑加入一条直线会对每个节点的“最优直线”产生怎样的影响，我们先将待插入直线拆分成若干个子区间 $[l_1, r_1], [l_2, r_2], \dots, [l_k, r_k]$ 。然后依次考虑这些区间，对于 $[l, r]$ ，设待插入直线为 l_1 ，原来的直线为 l_2 ， $mid = \lfloor \frac{l+r}{2} \rfloor$ ，那么分情况讨论：

- 如果 l_1 在 l, r 处的取值均 $\geq l_2$ ，那么直接替换并返回即可。
- 如果 l_2 在 l, r 处的取值均 $\geq l_1$ ，那么说明 l_1 没有用，直接返回即可。
- 如果以上两种情况均不成立，那么我们考虑两条直线在 mid 处的取值的大小关系：
 - 如果 l_1 大一些，那么如果 l_1 在 l 处的取值 $> l_2$ ，说明 l_2 不可能成为左边点的最高直线，直接把当前节点的最优直线换成 l_1 ，然后递归向右区间内插入 l_2 即可。否则则递归向左区间插入。
 - 否则，那么如果 l_1 在 l 处的取值 $> l_2$ ，那么说明 l_1 不可能成为右边点的最高直线，递归向左区间插入 l_1 ，否则递归向右区间插入 l_2 。

由于每条直线作为最优区间的深度在不断递增，因此可以根据这个作为推算李超线段树时间复杂度的方法。如果是全局插入那么插入一个直线只会带来 $\log$ 的势能，因此全局插入复杂度 $1\log$ 。如果是区间插入那么一个 $[l_i, r_i]$ 就会带来 $\log$ 的势能，因此区间插入复杂度 $2\log$ 。

值得注意的是，李超线段树并不只能用来维护直线凸包，事实上，如果两个曲线最多只有一个交点，那么也可以用李超线段树来维护这样的曲线。并且插入删除复杂度都同上。

李超线段树并不支持随机删除一个已经加入的直线，但其支持撤销，因此可以配合线段树分治 / DFS 栈使用。

5.2.3. 势能分析线段树与 ODT

所谓势能分析线段树，就是一些看似复杂度不正确的操作，但是我们通过分析一些形如“每经过一个节点 xxx 就会减少”，并通过分析初始时刻该量的总和不会特别大来得出每个节点被经过次数之和不会很大的结论。

势能线段树可以解决的第一类问题是使得数的数量级减少得很快的区间操作，如区间开平方、区间取模等操作，具体方法是我们记录一个区间最大值，然后进行区间修改时先考察这段区间内是否所有数都变成了 1 / 不能进行操作，如果是则返回，否则继续向下递归，递归到叶子节点时就进行相应的操作，这样根据势能分析，一个点最多被操作 $\log$ 次。总复杂度 $n \log n \log V$ 。

对于平衡树而言，其实也有一个类似的势能分析模型，这里顺带着提一下。大概就是如果我们在线地支持我们要对 $\geq k$ 的数都减去 k 这样的操作，考虑如何快速维护：我们以值为键值建出平衡树，每次进行操作的时候就将树分成三部分： $< k$, $[k, 2k]$ 和 $> 2k$ ，对于中间的部分，我们暴力将每个数减去 k 并插回第一部分中，然后在第三部分上打上 $-k$ 的标记并直接与第一部分合并。考虑这样做的正确性，由于第二部分的数量每进行一次 $-k$ 大小至少减半，而第三部分与第一部分合并时值的顺序不会改变，因此复杂度 $n \log^2 n$ 。

势能线段树可以解决的第二类问题是区间取 $\min$ ，区间求和。具体方法是记录区间 $\max$ 和严格第二大的值 $smax$ ，遍历到一个区间时，如果 $v > \max$ 则直接返回，如果 $v > smax$ 则打一个 $-(\max - v)$ 的标记表示将所有最大值都减去 $\max - v$ ，额外记录一个最大值个数即可。否则暴力递归子区间。注意到，对于一个区间而言，如果对其进行第三种操作，那么区间中不同数的种类

数会至少减少 1，因此每个节点被遍历次数总和是 $1\log$ 级别的。

当然还有一个也能处理这种势能分析，但和线段树毫无关系的数据结构：ODT，也顺带着在这里一并复习了。ODT 最大的优点是能够非常方便地处理区间赋值问题。大致思想就是，用 set 维护值域相同的连续段，每次区间赋值就暴力删去 $[l, r]$ 中零散的连续段并往 set 中插入一个大的，由于每个连续段只会恰好在插入和删除的过程中各被删除一次，所以总复杂度 $n \log n$ 。但是 ODT 的一个缺点是如果查询也要遍历区间中所有连续段那复杂度就不对了，因此 ODT 题常配合线段树，每次区间赋值就对连续段进行整体的操作。当然有的数据随机的题目也可以尝试使用 ODT，因为区间查询时你期望不会访问到的连续段的个数不会太多。

5.3. 根号算法

5.3.1. 根号分治

对于有些题目，特别是出现“出现次数”或者“集合大小总和是定值”等字眼的题目，如果我们发现，当集合大小（或者一个数的出现次数）比较小的时候，我们可以采取 $O(|S|)$ 的暴力，当集合大小比较大的时候，这样的集合个数不会很多 ($O(\frac{n}{|S|})$ 个)，我们可以枚举所有这样的集合并一一计算它们的贡献，此时我们就可以将二者结合起来，当 $|S| \leq \sqrt{n}$ 时采取第一种维护方法，当 $|S| > \sqrt{n}$ 时采取第二种。这样可以在 $O(\sqrt{n})$ 的时间内计算这些的集合的贡献之和。

对于根号分治，算法性的知识不多，更多地是分析具体题目培养感觉。

5.3.2. 分块

分块的大致思想是，将序列分为 $\sqrt{n}$ 个大小为 $\sqrt{n}$ 的部分。碰到单点修改时就暴力重构修改位置所在的块，碰到区间修改时就暴力重构左右端点所在块，而对于中间的块则暴力打标记。对于查询，中间部分的贡献则根据我们在修改时维护的整块的信息计算，边角块的信息则暴力加上。

那么问题就来了，我们为什么要设计分块这个算法，或者说，传统意义上的线段树等维护区间半群的算法有什么解决不了的问题吗？不难发现，分块的过程从各种角度上来说都像极了 2 层 $\sqrt{n}$ 叉的线段树。如果线段树合并两个节点时，信息可以在 $O(1)$ 时间内上推，那么我们完全不用分块，直接线段树维护复杂度就是对的，但是如果我们合并两个信息 A, B 时信息只能在 $O(|A| + |B|)$ 的时间内维护，此时如果使用线段树那推到最上面一层时信息长度就会达到 $O(n)$ ，太劣了。此时使用分块就可以将合并的信息大小限制在 $\sqrt{n}$ 级别，这样复杂度就对了。

分块还可以起到平衡复杂度的作用，具体来说，如果我们有 n 次单点修改和 $n\sqrt{n}$ 次区间查询，那如果使用线段树复杂度将会变为 $n\sqrt{n} \log n$ ，但如果使用 $O(\sqrt{n})$ 单点修改 $O(1)$ 区间查询的分块，则复杂度将会较好地控制在 $O(n\sqrt{n})$ 范围内。

此外，除了对序列分块之外，还有一类很常见的分块技巧是对操作序列的时间轴进行分块，具体来说我们将操作序列分成 $\sqrt{q}$ 块，每次处理完一个块的修改时就暴力重构整个数据结构的信息，然后对于上一次重构到当前时间点内的所有修改，我们就暴力计算它们的贡献。这种 trick 能够处理那些一次修改对询问的贡献可以在很快时间内求出，且不强制在线的数据结构题。

最后稍微总结一下几个分块常用于求的问题：

- 与众数有关的大部分问题（有众数一般不能 polylog 。除非是严格众数）。
- 与区间逆序对有关的问题。
- 与区间内颜色种类有关的问题。

5.3.3. 莫队

对于一些数据结构题，如果每次询问我们要统计区间内的某个信息，**允许离线**。并且我们可以快速计算插入/删除一个元素时答案的变化，我们就可以采用莫队。其大致思想是维护两个指针 l, r ，然后将询问进行合理地排序，使得两个指针移动距离之和控制在根号级别。具体排序方法是，先分块，如果左端点不在同一块则按左端点大小排序，否则按右端点大小排序。考虑这样做的复杂度，对于左端点在同一块内的相邻询问，左端点最多移动 $\sqrt{n}$ ，右端点对于这样极长的连续段的移动次数总和是 $O(n)$ 的，对于左端点在不同块的相邻询问，左右端点都最多移动 n ，但这样的询问最多 $\sqrt{n}$ 个。因此左端点总移动次数 $(n+q)\sqrt{n}$ ，右端点总移动次数 $n\sqrt{n}$ ，总复杂度 $(n+q)\sqrt{n}$ 。插入/删除的复杂度。

莫队最基础的应用就是这些，当然莫队还有两个变种：带修莫队和回滚莫队。这里一并讲掉。

带修莫队

顾名思义，就是支持修改的莫队。正确的处理方法是将时间轴也看作一维，这样总共是三维，然后跑三维莫队即可。对于 k 维莫队而言，对询问排序的方法是将每一维所在的块的编号写在一起看作一个 k 元组然后依据 k 元组的排序方法对询问进行排序。时间复杂度 $n^{\frac{2k-1}{k}}$ ，对于 $k=3$ 而言就是 $k^{\frac{5}{3}}$ 。

回滚莫队

如果加入操作可以进行得比较顺利，但是删除操作比较麻烦，可以考虑回滚莫队。具体来说，考虑按照左端点所在的块一块一块处理这些询问，处理左端点在 $[L_b, R_b]$ 中的那些询问时，将所有询问按右端点从小到大排序，然后对于右端点也在 $[L_b, R_b]$ 中的询问直接暴力，对于右端点在 $[L_b, R_b]$ 之外的询问，我们维护一个左端点指针 l 初始等于 R_b+1 ，维护一个右端点指针 r ，然后回答时右指针就顺着遍历一遍不断加入即可，左指针每次回答询问之前先调整到 R_b+1 ，这个可以通过撤销操作实现，然后向左移到 l 。这样就巧妙地避开了删除操作。

如果删除操作比较顺利但加入操作复杂度较高（如链表等结构），则也可以用类似的方法处理，具体方法就是将右端点从大到小排序，然后左指针初始等于 L_b ，右指针初始等于 n ，然后右指针顺着删过去，左指针同样每处理完一个询问就撤销即可，这样同样不用加入。

5.3.4. 莫队二次离线

一种能够将 $n\sqrt{n}\log n$ 的莫队变为优美的 $n\sqrt{n}$ 的 trick。由于直接讲可能有点玄乎，我们以区间逆序对数为例讲解莫队二次离线的过程。

可以发现，对于区间逆序对而言，当我们向右移动右指针时，相当于计算有多少个 $[l, r]$ 中的数 $> a_{r+1}$ ，而如果我们一段连续的移动左右端点的序列看作一次操作，那么显然操作总次

数是 $O(q)$ 的，因此我们只要求出每次”操作“会使得逆序对数变化多少，我们就可以求出每个时刻的逆序对数进而求出答案。现在考虑如何求这个增量，以右端点向右移为例，假设右端点从 r_1 移到 r_2 ，此时左端点为 l ，记 $f(l, r, x)$ 变为 $[l, r]$ 中有多少个 $> a_x$ 的数，那么相当于求 $\sum_{i=r_1+1}^{r_2} f(l, i-1, i)$ ，差分一下可以变成 $\sum_{i=r_1+1}^{r_2} f(1, i-1, i) - f(1, l-1, i)$ ，前面一部分就维护 $f(1, i-1, i)$ 的前缀和即可，后一部分我们在 $l-1$ 处加入三元组 (l, r_1, r_2) 然后扫描线，扫到 x 时候考虑所有二元组 (l, r_1, r_2) ，由于 $\sum r_2 - r_1$ 是 $n\sqrt{n}$ 级别的，因此可以暴力枚举区间中所有位置，这样相当于 $O(n)$ 次单点加 $O(n\sqrt{n})$ 次区间查询。分块维护即可。

5.4. 数据结构的常用维护方法

5.4.1. CDQ 分治

对于一类数据结构题，如果我们需要支持动态加入 / 删除（如果需要删除，那么需要信息满足可删除性），并支持在插入的同时询问当前时刻的某个信息，且该问题的静态问题有较好的离线方法，我们就可以在 $O(\text{静态问题时间复杂度}) \cdot \log Q$ 的时间复杂度内离线解决原问题。

具体方法是：考虑对时间轴分治，分治到 $[l, r]$ 时记 $mid = \frac{l+r}{2}$ ，然后我们考虑 $[l, mid]$ 中的修改对 $[mid+1, r]$ 的询问产生的贡献，这个就跑一遍静态问题即可。注意，由于归并排序的存在，如果静态问题的复杂度瓶颈在排序，那么可以考虑归并排序，这样复杂度可以变为除排序之外的复杂度 $+Q \log Q$ 。

5.4.2. 线段树分治

同样是一个离线算法，用来解决那些需要支持插入、删除、查询，但删除操作较难处理的题目。此时我们发现，每个元素活着的时间是若干个区间的并，此时我们可以以时间轴为下标维护一棵线段树，然后将对应的区间拆成 $\log$ 个线段树上的子区间并在线段树上进行一遍 DFS，DFS 到线段树上一个节点时就将保存在这里的所有信息都插入进数据结构，回溯时撤销，这样我们就成功将删除操作转化为了撤销操作，同时，复杂度多一个 $\log$ 。

注意，线段树分治是半在线的，即如果你需要在 i 时刻进行了一次询问，并需要通过询问结果插入新的信息，并且新的信息满足，其活着的时间区间的左端点 $> i$ ，那么这种情况同样可以通过线段树分治维护，就一边在线段树上 DFS 一边插入新的信息即可。

5.4.3. 整体二分

对于一类数据结构题，我们需要解决多组询问，并且解决每组询问我们需要二分。如果对于单组询问而言一次二分复杂度可能达到线性，但是如果我们要对每组询问都求其答案是否 $\leq mid$ ，有非常优秀的离线做法，那么此时我们就可以考虑整体二分。具体方法是我们分治时传三个参数 l, r, S 表示我们已经知道答案在 $[l, r]$ 中的询问集合为 S ，记 $mid = \lfloor \frac{l+r}{2} \rfloor$ ，然后将数据结构的状态调整到 mid 处，这样即可知道哪些询问在左边，哪些询问在右边，递归分治即可。分治层数是 $\log$ 级别的，因此每个询问最多移动 $\log$ 次，而 mid 指针移动次数也是 $n \log n$ 级别的，所以总复杂度为 离线 check 每组询问的答案是否小于等于一个定值 mid 的复杂度 $\cdot \log V$ 。

5.4.4. 猫树分治

众所周知，线段树上推时需要合并两个节点的信息。但是如果合并复杂度太高，但是**插入单点的复杂度很低**，我们就可以考虑猫树分治。具体方法是分治区间 $[l, r]$ 时，我们记 $mid = \frac{l+r}{2}$ ，我们计算出 $[l, mid]$ 每个后缀的信息，与 $[mid+1, r]$ 所有前缀的信息，这个就进行 $r-l+1$ 遍单点插入即可。然后考虑处理询问区间跨 mid 的询问，假设询问 $[L, R]$ 满足 $l \leq L \leq mid < R \leq r$ ，那么我们只需对 $[L, mid]$ 和 $[mid+1, R]$ 即可，这样只用进行一次合并，在某些时候可以优于进行多次合并的复杂度，譬如合并两个背包，如果要知道合并的结果的每一位的值则需 V^2 ，但如果只需知道合并后第 w 位的值则只需要 $O(V)$ 。这样复杂度就是 $O(n \log n \cdot \text{单点插入的复杂度} + Q \cdot \text{进行单次合并的复杂度})$ 。

5.4.5. 二进制分组

二进制分组可以解决的问题与 CDQ 分治很类似，但是其比 CDQ 分治强在可以**在线地解决问题**，其也有一些局限性。

对于一类问题，如果允许离线我们有很好的离线预处理的方法，但是题目要求我们强制在线，此时我们就可以考虑二进制分组。具体来说我们将插入的元素分为 popcount 个大小为 2 的幂的组，譬如如果现在插入了 11 个元素我们就分为大小为 8, 2, 1 的组，每组我们离线预处理，每次新增一个元素时我们就像 2048 一样一路合并上去，如在 11 个元素时插入一个元素我们就依次合并 1, 1 和 2, 2，得到 8, 4 的组。这样查询只需在 $\log$ 个数据结构中按照允许离线的方式进行查询即可。

6. DP 方向

过于简单的 DP 知识（譬如什么是状压、区间 DP、树形 DP）这里不再提及，NOI 考察的 DP 能力主要有两点，一是状态的设计，二是 DP 的优化。由于前者算法性的内容较少，更多地是考验选手的思维和推性质的能力，偏重实战，可能会放到杂项里总结。因此这里主要会讲一下 DP 的优化。

6.1. DP 的单调性优化

顾名思义，利用单调性优化 DP。

6.1.1. 单调队列优化 DP

单调队列优化 DP 可以优化形如 $dp_i = X_i + \min_{L_i \leq j \leq R_i} \{dp_j + a_j\}$ 的 DP（当然 $\max$ 也同理，以下统一为 $\min$ ），这里要求 X_i, a_j 均为常值数组，即 X_i 只与 i 有关， a_j 只与 j 有关，且 L_i, R_i 均单调不降。具体方法就是用单调队列维护 $dp_j + a_j$ 的最值，插入删除方式都同单调队列。这样 dp_i 的最优决策点可以从队首找。

单调队列优化 DP 的一个经典应用是优化多重背包，因为多重背包的 DP 状态转移式形如 $dp_{i,j} = \max_{k=0}^{a_i} dp_{i-1,j-kw_i} + kv_i$ 。如果我们对每个 $\text{mod } w_i$ 的剩余类分开来考虑，会发现转移点组成一个区间，且这个区间的两个端点都是单调的，因此可以单调队列优化。使用单调队列优化的多重背包复杂度为 $O(nV)$ 。

6.1.2. 斜率优化

斜率优化可以优化形如 $dp_i = X_i + \min_{j < i} \{K_j V_i + B_j + dp_j\}$ 的 DP 转移式。有关这个式子可以有两种理解方式，一是可以将其看作有若干条直线 $k_j x + b_j$ ，我们要求这些直线与 $x = V_i$ 的交点的纵坐标的最小值。二是可以看作平面上有 n 个点 $(K_j, B_j + dp_j)$ ，我们要对每个点都做一条斜率为 $-V_i$ 的直线，要求其于 y 轴交点纵坐标的最小值。二者的区别在于前者一般要求直线凸包，后者一般要求点凸包，个人倾向于后者。

以后者的理解方式为例。那么相当于我们要维护一个下凸壳，最优决策点显然就是用 $k = -V_i$ 的直线去截这个凸包时第一个截到的点。现在问题是如何维护这个凸壳。如果所有点的横坐标，也就是 K_j 是单调递增的，那么做法是维护一个栈存储当前凸包内所有点，每当我们加入一个点 P 时，我们考察 P 与 $stk[tp]$ 连成的直线的截距，由于是下凸壳，如果 P 与 $stk[tp]$ 连成直线的截距 $< stk[tp]$ 与 $stk[tp-1]$ 连成的直线的截距，说明 $stk[tp]$ 不在下凸壳上，直接弹掉即可。由于每个元素最多被弹一次，所以建凸壳复杂度线性。接下来是找直线，如果 V_i 也是单调的，那么我们可以直接弹出队首元素直到队首元素和队列第二个元素形成直线的斜率 $> -V_i$ 。总复杂度线性。

上述讨论是基于 K_j 和 V_i 均是单调而言的，当然更多情况下 K_j 和 V_i 并不是单调的。怎么办呢？下面将讨论 K_j 和 V_i 不一定单调的情况：

- 如果 K_j 单调 V_j 不单调，那么需要在凸壳上二分斜率为 $-V_i$ 的位置。
- 如果 K_j 不单调，那么需要李超线段树 / CDQ 分治维护凸壳。李超线段树的方法是按照第一种思路维护 $k_j x + b_j$ 的最值。CDQ 分治的做法是考虑半在线地求这里的 dp_i ，每次分治时考虑左边的 j 对右边的 i 的贡献，这是一个静态的问题，将对应的 k 排序以后采用 K_j 单调的做法即可。

6.1.3. 决策单调性优化 DP

对于形如 $dp_i = \min_{j < i} dp_j + w(j+1, i)$ 的 1D1D 转移式，或者形如 $dp_{i,j} = \min_{k < i} dp_{i-1,k} + w(k+1, j)$ 的 2D1D 转移式，如果 $w(l, r)$ 满足四边形不等式：若 $a < b < c < d$ ，那么 $w(a, d) + w(b, c) > w(a, c) + w(b, d)$ （即相交由于包含），那么 dp 数组的决策点是单调的。这种性质被称为决策单调性。

接下来讲一下决策单调性的实现方法，一般来说有两种：分治和二分队列。

对于 2D1D 的决策单调性而言一般可以采取分治法找最优决策点。其流程是，分治参数里传五个参 i, l, r, ql, qr 表示现在在求第 i 层的 DP 值（也就是 $dp_{i,*}$ ），我们现在求 $j \in [l, r]$ 的最优决策点，我们可以肯定这些 $dp_{i,j}$ 的最优决策点都在 $[ql, qr]$ 中。然后求 $mid = \frac{l+r}{2}$ 处的最优决策点 x ，然后将决策点区间劈成 $[ql, x], [x, qr]$ 即可。

对于这种处理方法，一个挺 nb 的扩展之处在于，如果 $w(l, r)$ 要像莫队一样使用移动指针的方式求（譬如众数出现次数），那么复杂度也是对的，指针总移动次数可以被证明是 $n \log n$ 的。

对于 1D1D 的决策单调性，我们一般采取二分队列法找最优决策点。具体流程是，考察两个决策点 $x, y (x < y)$ ，那么必然存在一个时间点 i 满足 i 之前 x 比 y 优， i 之后 y 比 x 优，我们记这个时间点为 $f(x, y)$ ，那么我们考虑维护一个单调队列 q 满足队列中的元素都是某个时刻的最优决策点，加入一个元素 i 时，如果 $f(q[tl-1], q[tl]) > f(q[tl], i)$ 说明 $q[tl]$ 在比 $q[tl-1]$ 更优之前就不如 i 了，这说明 $q[tl]$ 已经没有用了，直接弹出队尾即可。查询 i 的最优决策点时，直接弹出队首直到 $f(q[hd], q[hd+1]) > i$ 即可。计算 f 需要二分，总复杂度 $n \log n$ 。

6.1.4. wqs 二分

wqs 二分常用来处理限制取物品数量为 k 的 DP 问题。具体来说，对于一些强制要求选 k 个物品，求最优代价的问题，如果我们记 $f(i)$ 表示选 i 个物品的最优代价。那么如果我们发现 $f(i)$ 是凸函数，求解 $f(k)$ 可以考虑以下思路：二分一个斜率 K ，然后用斜率为 K 的直线去截这个凸壳，根据截到的点的横坐标与 k 的关系判断应该调大斜率还是调小斜率，如果刚好截到 k 就直接返回。接下来考虑怎样计算用斜率为 K 截这个凸包会截到哪个点，不难发现这相当于每选一个物品会有 $-K$ 的代价，要最大化 / 最小化总代价。这样相当于没有物品个数限制，跑没有物品个数限制的做法即可。

具体实现时有些细节。wqs 二分的一个很重要的细节是，如果出现三点共线时，一定要**强制要求在代价相同时，比较选的物品个数（返回物品个数最大或最小皆可，取决于实现细节）**，因为这样对于凸壳上一段连续的斜率相同的连续段，我们二分到这个斜率时，才可以返回这段连续段的某个端点，有利于我们更好地判断是否截到了 k 所在的连续段。具体实现时，可以用一个 pair 或结构体储存代价和个数的二元组。

6.2. 矩阵优化 DP

矩阵这个知识点除了在线性代数中常考之外，在优化 DP 方面同样是一个很常见的考点。

6.2.1. 矩阵优化线性递推式 DP

众所周知，矩阵可以作为一种优化线性递推的手段，即对于 $f_i = \sum_{k=1}^n f_{i-k} g_k$ 的式子，我们可以用矩阵在 $n^3 \log m$ 的时间内求解一个 f_m 的值（当然有更高端的方法，但是不属于我们讨论的范畴）。因此有线性递推形式的 DP 也可以使用矩阵优化。

另一种可以用矩阵优化的 DP 形如 $dp_{i,j} = \sum_k dp_{i-1,k} \cdot A_{j,k}$ ，很明显这个式子就很像矩阵乘法，自然可以矩阵乘法优化。值得注意的是，很多图 / 自动机上走 k 步的路径的 DP 式子就是这个形式，因此很多图 / 自动机上走路的 DP 都可以用矩阵乘法优化。

使用矩阵乘法时有以下几个小 trick：

- 如果上面的 $\sum$ 中带有常数项，一个 trick 是矩阵的长宽各加 1 存放常数项。
- 一般来说，我们用到的广义矩阵乘法，如 $C_{i,j} = \min_k A_{i,k} + B_{k,j}$ 这样的 $(\min, +)$ 矩阵乘法也是满足结合律的，因此碰到这样形式的式子也可以考虑矩阵乘法。

6.2.2. 动态 DP

对于一类问题，如果我们要进行若干次区间询问，而每次区间询问的答案需要 DP，并且 DP 的转移可以用矩阵来刻画，此时就可以考虑用线段树维护矩阵乘法来做到高效进行区间查询，时间复杂度 $(n+q)\log nw^3$ ，其中 w 为矩阵大小。一个 trick 是可以在查询时使用向量乘矩阵而不是矩阵乘矩阵，这样时间复杂度可以降到 $n\log n \cdot w^3 + q\log n \cdot w^2$ 。这种 trick 被称为动态 DP (Dynamic Dynamic Programming, 简称 DDP)。一些经典的可以用 DDP 解决的问题形如区间本质不同子序列个数（值域很小）。值得注意的是，DDP 中用到的线段树也是支持修改的，不论是单点修改矩阵还是区间矩阵赋值都是可以快速维护的，所以 DDP 这种思想有很多可以扩展延伸的地方。

DDP 另一个很常考的扩展是将序列放到树上。常见的考法是一些比较基础 / 式子看起来比较简单的树形 DP，在上面套上修改，并需要支持对每次修改都动态维护整棵树的答案的变化。以树上最大独立集为例讲解下 DDP 的使用方法：首先我们有 DP 转移 $dp_{u,0} = \sum_{v \in \text{son}_u} \max(dp_{v,0}, dp_{v,1})$, $dp_{u,1} = a_u + \sum_{v \in \text{son}_u} dp_{v,0}$ 。考虑对树进行树链剖分，即 $light_{i,0/1}$ 分别表示 i 的轻儿子的 u 的 $dp_{u,0}$ 和 $\max(dp_{u,1}, dp_{u,1})$ 的和。那么有 $dp_{u,0} = \max(dp_{wson_u,0}, dp_{wson_u,1}) + light_{u,1}$, $dp_{u,1} = dp_{wson_u,0} + light_{u,0} + a_u$ 。如果我们定义两个矩阵 A, B 进行 $A \times B$ 的结果 C 满足 $C_{i,j} = \max_k A_{i,k} + B_{k,j}$ 。那么有 $\begin{bmatrix} dp_{u,0} & dp_{u,1} \end{bmatrix} = \begin{bmatrix} dp_{wson_u,0} & dp_{wson_u,1} \end{bmatrix} \times \begin{bmatrix} light_{u,0} & light_{u,0} + a_u \\ light_{u,1} & -\infty \end{bmatrix}$ 。于是我们能够用重儿子的 dp 值求出当前点的 dp 值。这样一来，对于一个点 u 而言，将 u 所在重链底到 u 这条路径上的路径乘起来就可以得到 u 的 DP 值。接下来考虑单点修改修改产生的影响，根据重链剖分的性质，从 u 到根节点路径上只会经过 $\log$ 条重链，也就是说，只有 $\log$ 个点的 $light_u$ 会发生影响，直接暴力修改这些点的 $light_u$ 并在线段树上修改它们的转移矩阵即可。

总结一下，单点修改的过程如下：

- 从 u 开始跳重链，每跳到一个链顶节点 v 就算出 v 的 DP 值并修改 fa_v 的 $light$ ，撤销掉 v 的 DP 值对 fa_v 的 $light$ 值的贡献。
- 修改 u 的转移矩阵。
- 再次从 u 开始跳重链，每跳到一个链顶节点就重新算出 v 的 DP 值，加入 v 新的 DP 值对 fa_v 的 $light$ 值的贡献并修改 fa_v 的转移矩阵。

注意点：

- 注意 **pushup** 时矩阵乘法的顺序，由于是从链底乘到链顶，所以应该是先乘右儿子后乘左儿子。
- 如果结构体内显示地维护一个 $w \times w$ 的数组表示矩阵，那么常数有可能会很大，优化的方法是维护几个变量表示矩阵中的关键元素。

使用树剖实现的 DDP 时间复杂度为 $n\log^2 n$ 。可以使用全局平衡二叉树实现的 DDP 时间复杂度为 $n\log n$ ，不过一般不会有出题人卡树剖，所以树剖的 $n\log^2 n$ 对于大部分题目而言已经足够了。

6.3. 其他 DP 有关的 trick

6.3.1. DP 套 DP

所谓 DP 套 DP，就是将 DP 的值压入 DP 状态中。如果把内层 DP 的状态看作图上的一个个节点，那么状态与状态之间的转移则可以视作一个自动机的模型，由此进行 DP 即可。

通常来说 DP 套 DP 的状态数不会太多，否则时间复杂度无法接受。一般来说在写不确定状态数的 DP 时可以写个爆搜找到所有有用的状态，有时真正有用的状态会比你想象中的少很多。

DP 套 DP 本身不常考 (upd after NOI2022, 我是小丑)，不过作为一个思想而言还是很有用的。

6.3.2. 整体 DP 与长链剖分优化 DP

见第三章 3.5。

6.3.3. 轮廓线 DP 与插头 DP

轮廓线 DP 与插头 DP 都可以用来解决数据范围很小（大约 $6 \sim 15$ 左右，视题目要求而定，有的较为麻烦的题需要记录的题较多数据范围可能会小一些）的网格上的状压 DP，再往上基本上很难跑过去了。因此碰见形如 $nm \leq 50, nm \leq 100, nm \leq 150$ 的数据范围，分别暗示着 $\min(n, m) \leq 7, 10, 12$ ，此时就可以考虑轮廓线 DP / 插头 DP。

对于不少网格上的状压 DP，一个很经典的转移方法是逐行转移，即对于每一行我们枚举上一行的 DP 状态和这一行的决策来推到下一行的 DP 状态。但是对于有的 DP 题，使用逐行转移的方式进行 DP 复杂度就很劣了。举一个简单的例子，给定一个由空格和障碍物组成的网格，问其能够最多放多少个 2×2 的正方形使得它们之间两两之间都没有公共区域。此时按照逐行转移的方法是，DP 状态里保存一个三进制数表示每一列最下面的部分有多少个空格，转移就枚举最下面一行哪些位置放了正方形，这样状态复杂度 $n3^m$ ，转移复杂度 $2^{m/2}$ 。考虑怎么将转移复杂度变成 $O(1)$ ：我们将“逐行转移”变成“逐格转移”，这样访问到一个点 (i, j) 时，我们 mask 里保存对于每一列而言，在以 (i, j) 为分界点的轮廓线上方有多少个空格没有被任何一个 2×2 的正方形覆盖，这样对于一个方案而言，我们只需在我们选择的所有 2×2 的矩形的右下角的位置加入该矩形的贡献，这样转移可以 $O(1)$ 进行，总复杂度 $nm3^m$ 。由于这种转移里涉及轮廓线，所以被称为“轮廓线 DP”。

对于一类与网格图连通性有关的轮廓线 DP，在进行轮廓线 DP 时我们需要记录与“插头”有关的信息，这类 DP 我们称为插头 DP。以插头 DP 的经典题：计算带有障碍的网格图的哈密顿回路为例，我们定义一条轮廓线的“插头”为哈密顿回路中穿过这条轮廓线的部分，显然如果我们从这个轮廓线的位置把哈密顿回路切成两半，那么显然插头是两两匹配的，因此我们考虑轮廓线 DP 时，用最小表示法记录这些插头的两两匹配情况。插入一个有障碍物的格子时，只有当这个点的上插头和右插头都不存在时才可以转移，插入一个无障碍物的格子时，与这个相连的哈密顿回路有 6 种连接方式，对这 6 种连接方式分别分类讨论即可。插头 DP 理论可达的状态可能很多，但是实际有用的状态数并不多。此时可以用哈希表储存有用的状态，这样效率会高不少。还有一点就是在实现最小表示法时，不必要爆搜出所有的状态再给它们重标号，一种简洁的方法是假设 $m+1$ 个插头的编号分别为 $a_1, a_2, \dots, a_{m+1}$ ，直接将这个数组进行压缩压成一个二进制数

放到哈希表里即可。进行二进制压缩时我们一般选择不小于插头种类数的最小的 2 的幂作为底数，譬如如果有 4 种插头就以 4 作为底数，有 5 或 6 种插头都以 8 作为底数。

一般来说，插头 DP 的细节都比较多，写的时候要格外注意。

7. 数论方向

7.1. gcd 相关基础知识

7.1.1. 裴蜀定理

不定方程 $ax + by = c$ 有解当且仅当 $\gcd(a, b) \mid c$ ，必要性显然。充分性可以通过扩展欧几里得算法的过程说明，将会在下面提及。

此定理还有一个延伸定理：对于一张 n 个点的图，对于每个 $0 \leq i < n$ 连边 $i \rightarrow (i + d) \bmod n$ ，那么最终会形成 $\gcd(d, n)$ 个环，每个环环长为 $\frac{n}{\gcd(d, n)}$ 。因为一个点 i 能走到 $(i + A) \bmod n$ 当且仅当 $dx + ny = A$ 有整数解。由裴蜀定理可以得出 $\gcd(d, n) \mid A$ 。使用 polya 定理时经常需要这个定理。

7.1.2. exgcd

考虑求出 $ax + by = \gcd(a, b)$ 的一组解，显然求出这组解后将 x, y 都乘以 $\frac{c}{\gcd(a, b)}$ 就可以得到 $ax + by = c$ 的解。

考虑效仿求 gcd 的辗转相除法，显然 $b = 0$ 时 $x = 1, y = 0$ 符合条件，我们假设 $bx_0 + (a \bmod b) \cdot y_0 = \gcd(a, b)$ ，那么可以得到 $bx_0 + (a - b \cdot \lfloor \frac{a}{b} \rfloor)y_0 = \gcd(a, b)$ ，即 $ay_0 + b(x_0 - \lfloor \frac{a}{b} \rfloor y_0) = \gcd(a, b)$ ，于是我们令 $x = y_0, y = x_0 - \lfloor \frac{a}{b} \rfloor$ 即可。如此递归下去即可求得一组解。

显然将 x 加上若干个 $\frac{b}{\gcd(a, b)}$ ， y 减去相应数量的 $\frac{a}{\gcd(a, b)}$ 之后等式依然成立，因此我们假设 x_0, y_0 为我们求得的特解，那么 x, y 的通解可以表示为 $x = x_0 + k \cdot \frac{b}{\gcd(a, b)}, y = y_0 - k \cdot \frac{a}{\gcd(a, b)}$ 的形式，其中 $k \in \mathbb{Z}$ 。

另外，可以证明我们求得的 x_0, y_0 满足 $|x_0| \leq \frac{b}{2\gcd(a, b)}, |y_0| \leq \frac{a}{2\gcd(a, b)}$ ($a = 0$ 或 $b = 0$ 或 $a = b$ 的情况可能要特殊处理一下)，证明可以归纳，这里不再赘述。这意味着我们求得的 x_0, y_0 不一定是正数但是绝对值不会太大。

7.1.3. 逆元

对于一个数 a ，定义其在 $\bmod p$ 的意义下的逆元为 $x (1 \leq x < p)$ ，当且仅当 $ax \equiv 1 \pmod{p}$ 。显然对于一个数 a ，其如果有逆元，那么其逆元一定是唯一的。而一个数有逆元的充要条件是 $a \perp p$ 。求一个数逆元的方法有两种，一是求方程 $ax + ny = 1$ 的解，exgcd 即可，二是若 p 是质数，则可以直接通过 a^{p-2} 求逆元。

7.2. 欧拉函数与欧拉定理

7.2.1. 欧拉函数

定义： $\varphi(n)$ 表示 $[1, n]$ 中与 n 互质的整数个数。

欧拉函数的性质如下，限于篇幅原因不一一证明：

1. 假设 n 的质因数分解的形式为 $n = p_1^{k_1} \cdot p_2^{k_2} \cdot p_3^{k_3} \cdot \dots \cdot p_m^{k_m}$ ，那么 $\varphi(n) = n \cdot \prod_{i=1}^m \frac{p_i - 1}{p_i}$ 。

特别地，对于质数 p ， $\varphi(p) = p - 1$ 。

2. (积性) 对于 $a \perp b$ 即 $\gcd(a, b) = 1$ ，有 $\varphi(ab) = \varphi(a)\varphi(b)$ ，满足该性质的函数被称为积性函数，如果把 $a \perp b$ 的前提去掉依然成立，那么该函数又被称为完全积性函数。

3. (重要性质) $\sum_{d|n} \varphi(d) = n$ ，证明大概就考虑所有分子在 $1 \sim n$ 中且分母为 n 的分数化简后的结果，设为 $\frac{a}{b}$ ，发现 $a \leq b$ 且 $a \perp b$ ，因此该分数会恰好对 $\varphi(b)$ 产生 1 的贡献，而又显然所有 $b | n, a \leq b$ 且 $a \perp b$ 的分数 $\frac{a}{b}$ 都会被统计。

4. (小结论) 对于 $n \geq 3$ ，有 $2 | \varphi(n)$ 。

5. 对于给定的 n 与 $d | n$ ，满足 $a \in [1, n]$ 且 $\gcd(a, n) = d$ 的 a 个数为 $\varphi(\frac{n}{d})$ 。

7.2.2. 欧拉定理

欧拉定理：对于 $p \geq 2$ 及 $a \perp p$ ，有 $a^{\varphi(p)} \equiv 1 \pmod{p}$ ，证明大概是构造 A 为 $[1, p]$ 中与 A 互质的集合， B 为 A 中每个数乘 a 后的结果，发现 A 与 B 在 $\text{mod } p$ 后的结果相同，但 B 的乘积是 A 的 $a^{\varphi(p)}$ 倍。

特别的，对于 p 为质数的情况，有 $\forall p \nmid a, a^{p-1} \equiv 1 \pmod{p}$ ，该定理被称为费马小定理。

扩展欧拉定理，对于任何 $p \geq 2$ ，都有 $a^p = \begin{cases} a^x & (x \leq \varphi(p)) \\ a^{x \bmod \varphi(p) + \varphi(p)} & (x > \varphi(p)) \end{cases}$ ，扩展欧拉定理

可以用来解决指数很大（譬如 10^{10^5} 之类的级别的数）。一个经典的应用是幂塔，譬如我们要求 $a_1^{a_2^{a_3^{\dots}}}$ ，那么显然要知道 a_1 指数上的东西，我们只要知道 $a_2^{a_3^{\dots}} \bmod \varphi(p)$ ，同理我们需要知道 $a_3^{a_4^{\dots}} \bmod \varphi(\varphi(p))$ ，以此类推，不难发现经过 $\log$ 层模数就会变成 1，因此幂塔的值 $\text{mod } p$ 只与幂塔最底下 $\log p$ 层有关，还有一个注意点是求快速幂不能直接 $\text{mod } \varphi(p)$ ，而要判断其是否 $> \varphi(p)$ ，如果是那么应当返回快速幂的值 $\text{mod } \varphi(p) + \varphi(p)$ ，否则就不用管。

7.3. 离散对数

离散对数可以用来求 $a^x \equiv n \pmod{p}$ 的最小的 x ，或者宣布无解。

7.3.1. BSGS 算法

可以用来解决 $a \perp p$ 的情况。具体方法是根号分治，我们设阈值 $B = \lceil \sqrt{\varphi(p)} \rceil$ ，然后将 $a^0, a^1, \dots, a^{B-1}$ 放入哈希表中，然后枚举 $i \in [0, B-1]$ ，查看 $n \cdot a^{-iB}$ 是否在哈希表中，如果在则直接返回即可。时间复杂度 $\sqrt{\varphi(p)}$ 。

如果要对同一个 p 求 n 次 BSGS，则可以将阈值设为 $\sqrt{n \cdot \varphi(p)}$ ，根据根号平衡的思想知这是最优复杂度。

当然，对于特殊的 n （比如 ± 1 ），求解上述方程的解可以有更优的复杂度，这个我们后面再

说。

7.3.2. 扩展 BSGS 算法

可以扩展到 $a \perp p$ 的情况。

记 $f(a, n, p)$ 表示最小的 x 使得 $a^x \equiv n \pmod{p}$ 。假设 $d = \gcd(a, p)$ ，特判掉 $n = 1$ 的情况，此时直接返回 0 即可（如果要求正整数可能还要加一些特判），否则显然其等价于 $\frac{a}{d} a^{x-1} \equiv \frac{n}{d} \pmod{\frac{p}{d}}$ （如果 $d \nmid n$ 那么显然无解），递归即可。

显然最多 $\log p$ 轮可以递归到 $a \perp p$ 的情况。

7.4. 线性同余方程组

线性同余方程组指形如

$$\begin{cases} n \equiv a_1 \pmod{m_1} \\ n \equiv a_2 \pmod{m_2} \\ \dots \\ n \equiv a_k \pmod{m_k} \end{cases}$$

的方程组。

7.4.1. 中国剩余定理 CRT

CRT 可以用来解决上述方程组中 m_i 两两互质。显然对于此种情况而言， $[0, \prod m_i - 1]$ 中恰好存在一个 n 符合该条件。那么可以证明， $\sum_{i=1}^k a_i b_i \prod_{j \neq i} m_j$ 为一个符合条件的 n ，其中 b_i 是 $\prod_{j \neq i} m_j$ 在 $\text{mod } m_i$ 意义下的逆元，证明大概就发现只有上述和式中第 i 项会影响和式 $\text{mod } m_i$ 的值，而由于 b_i 是 $\prod_{j \neq i} m_j$ 在 $\text{mod } m_i$ 意义下的逆元，所以上式的值 $\text{mod } m_i = a_i$ 。

7.4.2. 扩展中国剩余定理 exCRT

考虑用合并两个同余方程组 $n \equiv a_1 \pmod{m_1}$ 和 $n \equiv a_2 \pmod{m_2}$ 的思想解决问题，假设 $n = xm_1 + a_1 = ym_2 + a_2$ ，那么有 $xm_1 - ym_2 = a_1 - a_2$ ，exgcd 求解 x, y 即可，如果 $\gcd(m_1, m_2) \nmid a_1 - a_2$ 那么同余方程组无解。

7.5. Lucas 定理相关

Lucas 定理一般用于求解模数很小的组合数，此时无法预处理阶乘逆元来求组合数，因为阶乘逆元有可能不存在。

7.5.1. Lucas 定理

Lucas 定理：对于质数 p ，有 $\binom{n}{m} \equiv \binom{n/p}{m/p} \cdot \binom{n \bmod p}{m \bmod p} \pmod{p}$ 。

证明：对于质数 p ，根据组合数的计算式有 $\forall 1 \leq i < p, \binom{p}{i} \equiv 0 \pmod{p}$ ，因为分子上的

$p!$ 没得可消了。这样我们可以得到 $(1+x)^p \equiv 1+x^p \pmod{p}$ ，然后我们构造生成函数 $\binom{n}{m} = [x^m](1+x)^n = [x^m](1+x^p)^{n/p} \cdot (1+x)^{n \bmod p} = [x^{\lfloor \frac{m}{p} \rfloor p}](1+x^p)^{n/p} \cdot [x^{m \bmod p}](1+x)^{n \bmod p}$ ，化简一下可以得到上面的式子。预处理复杂度 $O(p)$ ，计算答案复杂度 $O(\log_p n)$ 。

7.5.2. 扩展 Lucas 定理

扩展 Lucas 定理和 Lucas 定理关系不大，但是鉴于其与 Lucas 定理都可以用来解决小模数组合数求值问题所以就将它们相提并论了。与 Lucas 定理不同的是，扩展 Lucas 定理不需要 p 是质数。

我们首先对 p 分解质因数，显然对于一个分解出来的 p^k ，如果我们能算出 $\binom{n}{m} \bmod p^k$ 的值，就可以通过 CRT 得出原来的 $\binom{n}{m} \bmod p$ 的值。接下来考虑如何求 $\binom{n}{m} \bmod p^k$ ，显然我们只要知道 $\binom{n}{m}$ 里 p 的次数，以及 $\binom{n}{m}$ 除掉所有 p 之后的结果就可以知道 $\binom{n}{m} \bmod p^k$ 。我们考虑设 $f(n, p)$ 表示 $n!$ 里 p 的出现次数， $g(n, p)$ 表示 $n!$ 除掉所有 p 后得到的所有结果，于是 $\binom{n}{m} \equiv \frac{g(n, p)}{g(m, p)g(n-m, p)} \cdot p^{f(n, p)-f(m, p)-f(n-m, p)} \pmod{p^k}$ ，于是现在我们的任务是计算 f, g ，对于 f ，有 $f(n, p) = \lfloor \frac{n}{p} \rfloor + f(\lfloor \frac{n}{p} \rfloor, p)$ ，对于 g ，显然我们可以将 $[1, n]$ 中的数的贡献分为 p 的倍数和不是 p 的倍数计算，对于不是 p 的倍数的部分，显然贡献是若干个周期 + 一个零头，周期直接算了快速幂一下即可，零头也暴力算。对于是 p 的倍数的部分，发现统一除掉 p 以后贡献就是 $g(\lfloor \frac{n}{p} \rfloor, p)$ 。将两种贡献乘起来即可。

7.5.3. Kummer 定理

$\binom{n+m}{m}$ 中质数 p 出现次数就是 $n+m$ 在 p 进制下进行加法时的进位次数。证明可以通过 Lucas 定理说明。

应用该定理，可以快速判断组合数的奇偶性： $\binom{n+m}{m}$ 为奇数当且仅当 $n \& m = 0$ 。

7.6. 阶与原根

7.6.1. 阶

对于一个给定的数 p 与某个 a 满足 $a \perp p$ ，我们定义 a 在 $\bmod p$ 意义下的阶为最小的满足 $a^x \equiv 1 \pmod{p}$ 的正整数 x 。记作 $\delta_p(x)$ 。显然对于任意 $a \perp p$ 都存在阶，因为根据欧拉定理 $a^{\varphi(p)} \equiv 1 \pmod{p}$ 。而显然只有 $a \perp p$ 阶才存在。

阶有如下性质：

1. $\delta_p(x) \mid \varphi(p)$ ，否则 $a^{\varphi(p)} \equiv 1 \pmod{p}$ 不成立。
2. $x^a \equiv 1 \pmod{p}$ 当且仅当 $\delta_p(x) \mid a$ ，否则可以得出 $x^{\gcd(a, \delta_p(x))} \equiv 1 \pmod{p}$ ，与阶的最小性矛盾。

有了这两个性质考虑如何求阶，一种暴力是 BSGS，复杂度 $\sqrt{m}$ 太劣了，考虑更优的做法。另一种是试除法，即先对 $\varphi(p)$ 分解质因数，然后依次考虑每个质因子，在保证 $a^x \equiv 1 \pmod{p}$ 的

前提下不断令 x 除以这个质因子，这样复杂度 $\log^2 p$ 。同样的做法也适用于求解方程 $a^x \equiv -1 \pmod{p}$ 的最小整数解。

7.6.2. 原根

对于一个数 p ，一个数 a 是它的原根当且仅当 $a^0, a^1, a^2, \dots, a^{\varphi(p)-1}$ 互不相同。显然，原根的等价的定义是满足 $\delta_p(a) = \varphi(p)$ 的 a ，因为如果 $a^0, a^1, a^2, \dots, a^{\varphi(p)-1}$ 中存在相同的数，必然存在 $n < \varphi(p)$ 使得 $a^n \equiv 1 \pmod{p}$ 。

注意，并不是所有的数都有原根，只有 $2, 4, p^k, 2p^k$ 形式的数才有原根，其中 p 为奇质数， $k \in \mathbb{N}^+$ 。

接下来考虑如何判定一个数是否为 p 的原根，一种方法是直接暴力求阶，时间复杂度 $\log^2 p$ 。不过发现 $\delta_p(a) = \varphi(p)$ 等价于对于 $\varphi(p)$ 的所有真因子 n 都有 $a^n \not\equiv 1 \pmod{p}$ 。这样我们有一个等价的判定方法：考虑 $\varphi(p)$ 的所有质因子 p_0 ，检验是否有 $a^{\frac{\varphi(p)}{p_0}} \not\equiv 1 \pmod{p}$ ，如果是那么 a 不是 $\text{mod } p$ 意义下的原根，正确性显然，这样复杂度可以降至 $\omega(p) \log p$ 。

接下来考虑如何求出一个数的所有原根。首先是找出最小原根，根据著名结论，一个数如果存在原根，那么其最小原根是 $n^{0.25+\epsilon}$ 级别的，证明非常繁琐，这里略去。这样我们暴力枚举即可在 $n^{0.25+\epsilon} \cdot \omega(n) \log n$ 的时间复杂度内找到所有原根。接下来考虑如何求出所有原根，原根的最大好处是**可以将 $\text{mod } p$ 的简化剩余系中的乘法转化为 $\text{mod } \varphi(p)$ 意义下的加法，根据这个性质可以得出 $\delta_p(g^x) = \frac{\varphi(p)}{\gcd(\varphi(p), x)}$ ，其中 g 为 $\text{mod } p$ 意义下的原根。这样 g^x 为 $\text{mod } p$ 意义下的原根当且仅当 $x \perp \varphi(p)$ ，根据这个我们可以知道 $\text{mod } p$ 意义下的原根个数为 $\varphi(\varphi(p))$ ，均可以表示为 g^x 的形式，其中 $x \perp \varphi(p)$ ，这样我们可以求出 $\text{mod } p$ 意义下的所有原根。

7.7. 数论函数与莫比乌斯反演

7.7.1. 莫比乌斯函数

定义 $\mu(x) = \begin{cases} 0 & x \text{ 存在次数大于等于 } 2 \text{ 的因子} \\ (-1)^{x \text{ 不同质因子个数}} & \text{otherwise} \end{cases}$ 。

有关 $\mu(x)$ 最重要的性质是 $\sum_{d|n} \mu(d) = [n=1]$ ，因为发现上式等价于 $\sum_{i=0}^{\omega(n)} \binom{\omega(n)}{i} (-1)^i$ ，根据第四章中的结论 $\sum_{i=0}^n \binom{n}{i} (-1)^i = [n=0]$ 可得上述结论。

7.7.2. 数论函数

定义在 $\mathbb{N}^+$ 的函数称为数论函数。

一个数论函数 f 被称为积性函数，当且仅当 $f(1) = 1$ 且 $\forall x \perp y$ 都有 $f(xy) = f(x)f(y)$ 。

一个数论函数 f 被称为完全积性函数，当且仅当 $\forall x, y$ 都有 $f(xy) = f(x)f(y)$ 。

常见的积性函数与完全积性函数：

1. $\epsilon(x) = [x=1]$ ，完全积性函数。
2. $I(x) = 1$ ，完全积性函数。

3. $id(x) = x$, 完全积性函数。

4. $\varphi(x)$ 表示 $[1, x]$ 中与 x 互质的数的个数, 积性函数。

5. $\mu(x) = \begin{cases} 0 & x \text{ 存在次数大于等于 } 2 \text{ 的因子} \\ (-1)^{x \text{ 不同质因子个数}} & \text{otherwise} \end{cases}$, 积性函数。

6. $d(x)$ 表示 x 的因子个数, 积性函数。

7. $\sigma(x)$ 表示 x 的因子个数之和, 积性函数。

7.7.3. 狄利克雷卷积

对于两个数论函数 f, g , 定义它们狄利克雷卷积的结果 h 满足 $h(n) = \sum_{d|n} f(d)g(\frac{n}{d})$ 。容易证明狄利克雷卷积满足交换律、结合律, 同时两个积性函数的狄利克雷卷积也是积性函数。由于 $f * \epsilon = f$, 因此 ϵ 为狄利克雷卷积运算的单位元, 同时定义 f^{-1} 为满足 $f * f^{-1} = \epsilon$ 的数论函数。

很多重要的组合恒等式都可以从狄利克雷卷积的角度理解:

- $\sum_{d|n} \varphi(d) = n$ 可知 $\varphi * I = id$ 。
- $\sum_{d|n} \mu(d) = [n=1]$ 可知 $\mu * I = \epsilon$, 该式同时也说明 μ 与 I 互为狄利克雷卷积运算的逆元。
- 由 $\varphi * I = id$ 和 $\mu * I = \epsilon$ 也可以推出 $id * \mu = \varphi$, 具体方法是在 $\varphi * I = id$ 的两边各卷上一个 μ , 这样左边的 $\mu * I$ 抵消了, 左边只剩下 φ 了。

7.7.4. 整除分块

对于一个数 n 而言, $\lfloor \frac{n}{i} \rfloor$ 最多只有 $O(\sqrt{n})$ 种不同的取值。因为当 $i \leq \sqrt{n}$ 时总共只有 $\sqrt{n}$ 个数, 当 $i > \sqrt{n}$ 时 $\lfloor \frac{n}{i} \rfloor$ 最大也只有 $\sqrt{n}$, 因此最多 $2\sqrt{n}$ 个取值。

如果我们要找出所有 $\lfloor \frac{n}{i} \rfloor$ 的取值, 我们不必显式地把根号分治体现在程序中, 一种方法是维护两个变量 l, r , 初始 $l = r = 1$, 然后每一轮令 l 为上一轮的 r 加 1, r 为 $\lfloor \frac{n}{\lfloor \frac{n}{l} \rfloor} \rfloor$ 即可。

这样所有 $\lfloor \frac{n}{l} \rfloor$ 组成的集合就是 $\lfloor \frac{n}{i} \rfloor$ 的所有不同的取值。

整除分块可以用来计算形如 $\sum_{i=L}^R \lfloor \frac{n}{i} \rfloor f(i)$ 的和式, 计算方法就是对 $[L, R]$ 整除分块, 对于每一段 $\lfloor \frac{n}{i} \rfloor$ 相同的区间, 我们只需要知道它们 f 的和即可, 这个可以通过求出前缀和数组解决。对于形如 $\sum_{i=L}^R \lfloor \frac{n}{i} \rfloor \lfloor \frac{m}{i} \rfloor f(i)$, 也可以用整除分块解决, 只需要令上面过程中的 r 设为 $\min(\lfloor \frac{n}{\lfloor \frac{n}{l} \rfloor} \rfloor, \lfloor \frac{m}{\lfloor \frac{m}{l} \rfloor} \rfloor)$ 即可。

7.7.5. 莫比乌斯反演

莫比乌斯反演, 顾名思义就是运用莫比乌斯函数的性质将和式中的一些元素的形式进行转化, 使其变得易于快速计算。一般来说, 莫比乌斯反演的形式有三种:

- $\sum_{d|n} \mu(d) = [n=1]$, 此种形式的莫比乌斯变换一般可以化简形如 $[\gcd(i, j) = d]$ 的东西, 具体方法是先将其变为 $[\gcd(\frac{i}{d}, \frac{j}{d}) = 1]$, 再通过枚举 $p | \frac{i}{d}, p | \frac{j}{d}$, 并将 p 放到和式的第一个位置进行计算。譬如要求 $\sum_{i=1}^n \sum_{j=1}^n \gcd(i, j)^2$, 我们可以将式子先转化为 $\sum_{d=1}^n d^2 \cdot \sum_{i=1}^n \sum_{j=1}^n [\gcd(i, j) = d]$, 然后 $\sum_{d=1}^n d^2 \cdot \sum_{i=1}^{n/d} \sum_{j=1}^{n/d} [\gcd(i, j) = 1]$, 枚举 $p | i, p | j$ 可以得到 $\sum_{d=1}^n d^2 \cdot \sum_{p=1}^{n/d} \mu(p) \lfloor \frac{n}{dp} \rfloor^2$, 然后考虑枚举 $dp = T$ 有 $\sum_{T=1}^n \lfloor \frac{n}{T} \rfloor^2 \sum_{d \cdot p = T} d^2 \mu(p)$, 记 $f(i) = id_2 * \mu$, 那么这个式子的形式与 7.7.4 相同, 如果我们已经预处理出 f 的前缀和, 那么我们就可以 $\sqrt{n}$ 地求这个式子了。
- $f_n = \sum_{d|n} g_d \Rightarrow g_n = \sum_{d|n} f_d \mu(\frac{n}{d})$, 道理很简单, $f = g * I$, 两边同时卷上 μ 得 $g = f * \mu$ 。
- $f_n = \sum_{n|d} g_d \Rightarrow g_n = \sum_{n|d} f_d \mu(\frac{d}{n})$, 此式虽然没法写成狄利克雷卷积的形式, 但是可以效仿二项式反演的推法进行证明。

7.7.6. $f * I$ 的快速求法

对于已知的数论函数 f , 我们希望能快速求出 $g = f * I$ 的每一项的系数。一种很显然的方法是调和级数地枚举前缀和, 不过当 n 达到 10^7 级别, 特别 g 函数还带取模的时候效率会变得十分低下。我们考虑 $1 \sim n$ 中的所有质数 p_i , 如果我们对于每个数将其每个质数的次数写下来得到一个 $prime_num$ 维的向量, 那么我们的过程等价于高维前缀和。我们考虑枚举所有质数 p_j , 然后从小到大枚举 $i \in [1, \frac{n}{p_i}]$ 然后令 $g_{ip_j} \leftarrow g_{ip_j} + g_i$ 即可。这样复杂度是 $\sum_{x \in \text{prime}} \lfloor \frac{n}{x} \rfloor = n \log \log n$ 。

7.8. 积性函数前缀和的求法

回顾上面整除分块求解形如 $\sum_{i=L}^R \lfloor \frac{n}{i} \rfloor f(i)$, 我们将 $\lfloor \frac{n}{i} \rfloor$ 分成 $O(\sqrt{n})$ 段之后, 等价于我们需要对每一段 $[L, R]$ 计算 $\sum_{i=L}^R f(i)$, 这等价于我们要快速求出 $\sum_{i=1}^n f(i)$, 如果 n 比较小那么直接把 f 预处理出来后前缀和就行了, 否则就需要一些技巧了。

7.8.1. 线性筛

从 2 开始枚举所有数, 枚举到一个数 i 时从小到大枚举所有已经确定的质数 p_j 并将 $i \cdot p_j$ 标记为不是质数, 如果 $i \bmod p_j$ 就在筛完 $i \cdot p_j$ 后 break 掉, 这样显然每个数都会被其最小质因子筛去, 可以在线性时间内筛得所有质数。那么又该如何求 $f(i)$ 呢? 其实我们如果能在 $O(1)$ 时间内算出 $f(p^k)$, 我们就可以在 $O(n)$ 时间内筛出所有 $f(i)$, 具体方法是我们预处理 mn_i 表示 i 的最小质因子的次数, 这样 $f(i) = f(\frac{i}{mn_i}) \cdot f(mn_i)$, 直接扫一遍求出来即可。

优点: 好写。缺点: 太慢, 大部分筛的复杂度都是亚线性的, 因此与大部分筛的效率相比线性筛算很慢很慢的了。因此碰到 10^7 级别的筛果断选择线性筛。

7.8.2. 杜教筛

杜教筛可以在 $n^{2/3}$ 内对所有 n 的关键点（也就是可以表示为 $\lfloor \frac{n}{i} \rfloor$ 的点）求前缀和。前提存在一个狄利克雷卷积恒等式 $f * g = h$ ，其中 g, h 的前缀和我们能够高效地求出。具体来说流程如下：

$$\begin{aligned} & \sum_{i=1}^n h(i) \\ &= \sum_{i=1}^n \sum_{j|i} g(j) f\left(\frac{i}{j}\right) \\ &= \sum_{j=1}^n g(j) \sum_{i=1}^{n/j} f(i) \\ &= \sum_{j=1}^n g(j) s_f\left(\frac{n}{j}\right) \\ &= s_f(n) + \sum_{j=2}^n g(j) s_f\left(\frac{n}{j}\right) \end{aligned}$$

其中 $s_f(n)$ 表示 $\sum_{i=1}^n f(i)$ 。这样我们可以得到 $s_f(n) = \sum_{i=1}^n h(i) - \sum_{j=2}^n g(j) s_f\left(\frac{n}{j}\right)$ ，整除分块求后面一部分即可。加个记忆化搜索可以被证明是 $n^{3/4}$ 的，因为我们递归到的 $s_f(n)$ 肯定都是原来的 n 的关键点，而对于 n 的某个关键点 n_0 ，我们可以在 $\sqrt{n_0}$ 的时间内求其前缀和，因此总复杂度是 $\sum_{n_0 \text{ 是 } n \text{ 的关键点}} \sqrt{n_0} = n^{3/4}$ 。如果我们线性筛预处理 $n^{2/3}$ 以内的前缀和，那么复杂度可以被证明是 $n^{2/3}$ 的。具体证明类似，这里略去。杜教筛一般能够处理的规模的上限是 10^{10} 级别的。

优点：效率高、好写、可以求每个关键点处的前缀和。**缺点：**对积性函数的限制比较苛刻，必须要构造出相应的狄利克雷卷积的式子才可以杜教筛。

7.8.3. Powerful Number 筛

定义 Powerful Number 为每个质因子出现次数都 ≥ 2 的数，那么有一个结论是 $1 \sim n$ 中 Powerful Number 的个数是 $\sqrt{n}$ 级别的，因为所有 Powerful Number 都可以写成 $a^2 b^3$ 的形式，而 $a^2 b^3$ 形式只有 $\sum_{i=1}^{\sqrt{n}} \sqrt[3]{\frac{n}{i^2}}$ 个，积分一下可以得到 $O(\sqrt{n})$ 。

那么如何使用 Powerful Number 求积性函数前缀和呢？我们假设我们要求积性函数 f 的前缀和，那么我们考虑拟合一个积性函数 g 满足对于质数 p ， $g(p) = f(p)$ 且 g 的前缀和有比较好的求法（要么可以 $O(1)$ ，要么可以块筛），此时我们构造 $h = f * g^{-1}$ ，即 $g * h = f$ ，那么我们可以得到 $f(p) = g(1)h(p) + h(1)g(p) = h(p) + g(p)$ ，于是 $h(p) = 0$ ，又因为积性函数卷积积性函数、积性函数的逆还是积性的，故 $h(n)$ 只在 Powerful Number 处的取值非零，这样我们有 $\sum_{i=1}^n f(i) = \sum_{i=1}^n \sum_{j|i} g(j) h\left(\frac{i}{j}\right) = \sum_{i=1}^n [\text{i is powerful number}] \cdot h(i) \cdot s_g\left(\frac{n}{i}\right)$ ，直接 DFS 出所有 Powerful Number 后筛一下 g 的前缀和即可。如果 $g(x)$ 的前缀和可以 $O(1)$ 求那么复杂度

$\sqrt{n}$ ，否则复杂度为块筛 $g(i)$ 前缀和的复杂度。

优点：效率高，质数的 ≥ 2 的幂处的取值再奇怪，只要质数处的取值比较优美就可以 **powerful number** 筛。**缺点：**无法块筛，对质数处取值的要求比较苛刻。

8. 计算几何方向

8.1. 向量与向量的运算

一个向量是一个空间中带有方向的线段，与有向线段不同之处在于，我们不管向量在空间中的问题，只关心它的方向。这样我们可以用一个 k 元组表示一个 k 维空间中的向量。譬如 (x, y) 就可以表示 $(0, 0) \rightarrow (x, y)$ 的向量。向量之间存在加法和减法运算，其运算结果就是每一维相加 / 相减。向量的加减法遵循平行四边形法则。对于一个向量，定义其模长为它的长度，对于二维向量 $\vec{z} = (x, y)$ ， $|\vec{z}| = \sqrt{x^2 + y^2}$ 。

向量之间还存在点积和叉积。对于两个向量 $\vec{a} = (x_1, y_1), \vec{b} = (x_2, y_2)$ ，定义 $\vec{a} \cdot \vec{b} = x_1x_2 + y_1y_2$ ， $\vec{a} \times \vec{b} = x_1y_2 - x_2y_1$ 。两种运算的几何意义如下：

- 点积： $\vec{a} \cdot \vec{b}$ 的符号可以用来判定 $\vec{a}, \vec{b}$ 的夹角是否 $> \frac{\pi}{2}$ ，具体来说如果夹角 $< \frac{\pi}{2}$ 点积为正，如果大于则为负，如果恰好等于，即两向量垂直，那么 $\vec{a} \cdot \vec{b} = 0$ 。
- 叉积： $\vec{a} \times \vec{b}$ 的符号可以用来判定 $\vec{a}$ 的是否可以通过逆时针旋转得到 $\vec{b}$ ，如果 $\vec{b}$ 在 $\vec{a}$ 的顺时针方向则 $\vec{a} \times \vec{b}$ 为负，如果在逆时针方向则为正，如果共线则为 0。

除此之外，叉积的绝对值在数值上等于 $\vec{a}, \vec{b}$ 围成的平行四边形的面积。因此我们可以直接用 $\frac{1}{2}|(B-A) \times (C-A)|$ 计算 $\triangle ABC$ 的面积。叉积的另一个应用是不加分类讨论地求直线 AB 与 CD 的交点，具体方法是计算 $\triangle ABC$ 和 $\triangle ABD$ 的有向面积，记它们之间的比例为 $a:b$ ，那么设它们之间的交点为 E ，有 $CE:DE = a:b$ ，由此可以算出 E 的坐标。

在进行与向量有关的操作时，一个很重要的技能是将它们按照幅角排序，这项操作又被称为**极角排序**，可以通过调用 `atan2` 函数来实现。不少与点集有关的问题都可以通过极角排序使点变得有序。

8.2. 凸包

给定点集 S ，定义它们的凸包为它们最外层的点形成的凸多边形。感性理解就是将点集中的点视作钉子后，用一个无限大的橡皮筋去套这个点集后这个橡皮筋由于形变收缩后形成的区域。

求凸包有 Andrew 和 Graham 两种求法，二者效果等价，但是由于前者细节较少所以建议现场碰到时都实现前者。具体方法就是将所以点按 x 为第一关键字， y 为第二关键字进行排序，求一遍上凸壳求一遍下凸壳合并上来即可。求解上下凸壳的方法有点类似于斜率优化（见 6.1.2），不过由于有可能存在斜率相同的情况，所以既然知道了叉积的用法就尽量使用叉积来实现，以上凸壳为例，具体实现方法是维护一个栈，每次加入点 A 时查看是否有 $(stp[tp-1] - stk[tp]) \times (A - stk[tp]) \geq 0$ ，如果有则弹出栈顶元素，直到上述条件不成立后加入 A 。时间复杂度 $n \log n$ ，复杂度在于排序。

对于大部分题目，如果我们要对某个点集计算某个答案，在大多数情况下只保留凸包上的点

答案是不会发生变化的，此种情况下不管三七二十一我们不妨先求出凸包出来。另外，凸包上的点是有顺序而言的（逆时针 / 顺时针顺序），不像原点集中的点是杂乱无章的，因此有的题，建出凸包后我们可以像序列上的问题那样进行 two pointers / 二分 / 三分（如果二分 / 三分，建议先 check 一下是否满足可二分 / 三分性，很多凸包上的二分 / 三分都很容易假，具体例子可以见 UOJ 391 的 problem 6），此种算法又被称为旋转卡壳。

对于两个凸包 A, B 而言，可以证明， $\forall x \in A, y \in B, x + y$ 组成的多边形也是凸多边形，我们定义这个多边形为 A, B 的闵可夫斯基和，求两个凸包的闵可夫斯基和的方法很简单，直接把 A 中相邻两点之间形成的向量拎出来， B 中相邻两点形成的向量也拎出来后按极角归并排序即可。而显然闵可夫斯基和中最左下角的点就是 A, B 中最左下角的点加起来，这样我们容易恢复出原来的凸包。使用归并排序复杂度可以线性。

可以证明，两个凸包的闵可夫斯基和中的边数为 A, B 中相邻两点形成的不同斜率数，证明是容易的。

8.3. 最小圆覆盖

给定一个点集 S ，求半径最小的能够覆盖 S 中所有点的最小圆。

求解方法是考虑随机增量法，即将 S 中所有点随机打乱，然后依次考虑加入每个点的贡献，如果当前点 A 不在圆内，那么我们就先令圆为当前点，然后依次考虑 B 前面的所有点，如果它也不在圆内就将圆更新为 AB 的外接圆，再考虑前面的点 C ，如果 C 也不在圆内就将圆更新为 ABC 的外接圆。

乍一看复杂度不对。不过发现每个点被更新的概率实际上是 $\frac{3}{i}$ ，因此总复杂度为 $\sum_{i=1}^n i \cdot \frac{3}{i} = O(n)$ （期望）。

8.4. 半平面交

给定平面上 n 条直线，半平面交算法可以在 $O(n \log n)$ 的时间内求出这 n 条直线左侧的区域的交。

具体做法是，将所有直线按它们的辐角从小到大进行排序，对于辐角相同的直线，我们只保留最靠左的那一个。我们考虑维护一个双端队列保留现在所有有用的直线，这样，加入一条直线 l 时，我们考察队尾直线 $q[tl]$ 和 $q[tl-1]$ 的交点 A ，如果 A 在 l 的左侧那么说明 $q[tl]$ 是没有用的，直接删除队尾元素即可。同理也要对队首元素进行同样的操作。如果加入一条直线 l 时，我们发现 l 与 $q[tl]$ 平行，就意味着半平面交为 0，直接返回即可。最后我们还要考虑 $q[tl]$ 和 $q[tl-1]$ 的交点，如果在 $q[hd]$ 的左侧就弹出队尾。对队首也进行相应的操作。最后得到的队列中相邻两个直线的交点就组成了半平面交的顶点。

大部分半平面交题目需要手动添加边界。

对于那些符合某个条件的区域是一个半平面，要求符合所有条件的区域，就可以想到半平面交。

9. 网络流与二分图方向 (考察概率 30%)

9.1. 网络流与费用流

一个网络，就是一张有向图 $G = (V, E)$ ，其中每条有向边 (u, v) 上都有一个容量限制 $c(u, v)$ ，此外对于大部分网络，都存在两个特殊点 S, T ，分别称为源点和汇点（有源点和汇点的网络被称为有源汇网络，没有源点和汇点的被称为无源汇网络）。定义网络上的流函数 $f: (u, v) \rightarrow \mathbb{R}$ ，有以下两个要求：

- $f(u, v) \leq c(u, v)$ ，即每条边流量不能超过容量。
- 除了源点和汇点之外，每个点的流入量等于流出量，当然无源汇网络则要求所有点的流入量等于流出量。此条规则又被称为**流量守恒**。

对于有源汇网络，定义整张网络的流量为源点的流出量（也即汇点的流入量）。

定义一个网络的残量网络满足 $c'(u, v) = c(u, v) - f(u, v)$ ，如果 $c'(u, v) = 0$ 就将边 (u, v) 从残量网络中删去。简单来说就是将每条边的容量减去流量就可以得到残量网络的容量。

此外，对于部分网络，每条边上还有一个代价函数 $w(u, v)$ ，不管是否有源汇，定义整张网络的费用为 $\sum_{f(u, v) > 0} f(u, v)w(u, v)$ 。对于有源汇的网络，定义其最大流就是整张网络最大流量。对于有源汇费用流，定义其最小费用最大流为在流量最大的基础上的最小费用。

9.1.1. 最大流的求法

9.1.1.1. FF 算法与最大流等于最小割定理。 首先有一个很明显的错误解法：每次暴力 DFS，DFS 到一个所有边容量都 > 0 的路径就令 mn 为这条路径上所有边的容量的最小值然后将这条路径上所有边的容量都减去 mn ，很明显不对。考虑为什么这个做法不对，原因是因为我们可能会陷入一个不能继续增广、但不是最优的状态，这种情况我们其实可以撤销掉之前的某次增广后有更好的选择，因此我们考虑增加一个反悔机制：给每条边增加一个反向边 $v \rightarrow u$ ，初始方向边容量为 0，每次 $u \rightarrow v$ 流过 f 的流量就令 $v \rightarrow u$ 的容量加 f 。这样 DFS 时我们把反向边也算上即可。

因此我们得到一个非常暴力的做法：在带有反悔机制的图上 DFS 直到不能 DFS 为止。可以证明这样得到的就是最大流。

为什么加上反向边的反悔后暴力 DFS 直到不能 DFS 为止得到的流量就是最大流？我们先证明一个著名的结论：**最大流等于最小割**。具体来说，我们定义一个网络的一组割为一组满足 $A \cup B = V, A \cap B = \emptyset, S \in A, T \in B$ 的点集 A, B ，在此基础上定义割的权值为 $\sum_{x \in A, y \in B} c(x, y)$ 。那么证明最大流等于最小割，等价于证明以下两个命题：

1. 任意一组流的流量 $\leq$ 任意一组割的权值。对于一组割，我们记 $P = \sum_{x \in A, y \in B} f(x, y) - f(y, x)$ ，那么我们有对于任意一个流，它的流量都等于 P ，因为对于任意一条流必然是从 A 中的 S 开始，交替经过 A, B, A, B 最终到达 B 的序列，其经过 $A \rightarrow B$ 的边的个数必然比 $B \rightarrow A$ 的边数多 1，因此如果我们流过了 f 的流量，那么 P 会增加 f ，而网

流的流量显然也增加 f ，因此任意一个流的流量都等于 P ，而根据式子很明显可以看出

$$P \leq \sum_{x \in A, y \in B} c(x, y), \text{ 因此命题成立。}$$

2. **存在一组流的流量等于割**，显然最大流流量不会超过所有与 S 相连的边的容量之和，因此最大流必然存在。而如果某时刻残量网络上 S, T 不连通，我们令 A 为 S 在残量网络上可以到达的点， B 为 S 的补集即可，此外，我们还要说明这组割的

$$\sum_{x \in A, y \in B} c(x, y) = \sum_{x \in A, y \in B} f(x, y) - f(y, x).$$

由于此时 S, T 在有反悔机制的图的残量网络上不连通，所以不存在

在 $S \rightarrow T$ 的剩余容量 > 0 的边，因此对于 $x \in A, y \in B, x \rightarrow y$ 的边都满流，否则这条边剩余容量 $> 0, y \rightarrow x$ 的边都空流，否则反向边容量 > 0 ，这样 $f(x, y) = c(x, y), f(y, x) = 0$ ，有

$$\sum_{x \in A, y \in B} c(x, y) = \sum_{x \in A, y \in B} f(x, y) - f(y, x), \text{ 这样这组割的容量就等于此时的流量。}$$

结合上述两点即可知此时的流量 = 最大流 = 最小割。

9.1.1.2. EK 算法。 虽然 FF 算法的正确性有保障的，但是时间复杂度与流量有关，太劣了。考虑对这个算法进行一些改进，我们将暴力 DFS 改为找 $S \rightarrow T$ 的容量 > 0 的路径中长度最小的，这样可以证明复杂度是 nm^2 ，具体证明可以这里略去。

9.1.1.3. Dinic 算法。 事实上 EK 算法还可以进一步进行改进，改进方法是，我们求出残量网络上 S 到每个点的最短路 dis_x ，然后根据 dis 对整张图进行分层，此时从源点开始多路增广，具体来说 DFS 到一个点 x 时我们传两个参 x 和此时的流量 f ，每次增广时遍历 x 的所有满足 $dis_y = dis_x + 1$ 且 (x, y) 之间容量 > 0 的邻居 y 。设 z 为 x, y 之间边的容量，我们将 $\min(z, f)$ 的流量分配给 y 继续 DFS 即可。记 w 为将 $\min(z, f)$ 的流量分配给 y 后流到 T 的流量，我们直接令 f 减去 w ，答案加上 w ， (x, y) 边的容量减去 w ，反向边容量加上 w 即可。一个优化是类比欧拉回路，如果我们访问到一条边并且没有 return，说明这条边已经没有了用了，我们下一次访问到这个点时就不用访问这条边了，因此如果使用链式前向星存图，那么 head 数组在访问时可以像欧拉回路一样传引用，这种优化又被称为**当前弧优化**。

加当前弧优化、多路增广的 Dinic 算法复杂度为 n^2m ，证明不在此赘述。

虽说 Dinic 理论复杂度很高，但实际跑起来效率很高。对于一般图，Dinic 可以在 1s 内处理 $n = 10^4, m = 10^5$ 规模的图。

9.1.2. 最小费用最大流的求法

类比最大流，费用流的求法也需要建反边。显然反边的费用应该是原边的相反数。由于最小费用的限制，最小费用最大流不太好直接像最大流那样用多路增广为背景的 Dinic 算法，因此我们只能一条一条路增广。因此我们可以想到一个比较明显的贪心：每次取总费用最小的路径增广，由于有负权，找总费用最小的路径需要 SPFA。时间复杂度理论上界为 $O(nmf)$ ，但同样跑不满，一般可以处理 $n = 10^3, m = 10^4$ 规模的问题。对于 NOI 来说，网络流算法只需要了解 Dinic 和用 EK 实现的费用流，如果还是跑不过去那很有可能就是此题正解不是网络流。对于此种求最小费用最大流的方法，费用可以为负，但是不能出现负环（因为需要 SPFA）。如果出现负环则需要用上下界网络流的知识进行求解，在 9.2 章中会提及。

9.2. 上下界网络流

顾名思义，上下界网络流就是每条边还有一个属性 $l(x, y)$ 表示 $l(x, y) \leq f(x, y) \leq c(x, y)$ 限制了这条边流量的下界。上下界网络流的主要思想是**先为每条边分配 $l(x, y)$ 的流量，通过调整将其满足流量守恒，从而转化为无下界的网络流。**

下面是上下界网络流的一些经典模型和解决方法：

- 无源汇上下界可行流：我们需要构造出一张网络满足 $l(x, y) \leq f(x, y) \leq c(x, y)$ 且每个点都满足流量守恒定律。解决方法是先给每条边都预留 $l(x, y)$ 的流量，这样每条边流量就没有下界了，上界为 $c(x, y) - l(x, y)$ 。但是这样不一定满足流量守恒定理。具体来说预留 $l(x, y)$ 的流量就等价于给了 y 点 $l(x, y)$ 的流并使 x 点需要 $l(x, y)$ 。我们新建超级源汇 SS, TT ，连边 $SS \rightarrow y$ ，容量 $l(x, y)$ ， $x \rightarrow TT$ ，容量 $l(x, y)$ ，然后跑最大流，如果与 SS 相连的边没有全满流说明无解，否则最终 (x, y) 的流量等于建出的图中的 $f(x, y) + l(x, y)$ 。
- 有源汇上下界可行流：对于原图中的源汇 S, T ，连边 $T \rightarrow S$ ，容量 ∞ ，负责把 S 流到 T 的流退回，这样可以规约到无源汇的问题。
- 有源汇上下界最大流：求出一组可行流后，断掉 $T \rightarrow S$ 的边后**进行二次调整**，二次调整的方案是跑 $S \rightarrow T$ 的最大流，加上 $T \rightarrow S$ 的流量就是最大流。
- 有源汇上下界最小流：根据最小流是最大流的相反数这一结论，求出可行流后断掉 $T \rightarrow S$ 的边后，拿 $T \rightarrow S$ 的流量减去 $T \rightarrow S$ 的最大流就是最小流。
- 有源汇上下界费用流：只要将上述过程的最大流改为费用流即可。记得初始费用赋为 $\sum_{x,y} w(x, y)l(x, y)$ 。
- 有负环的费用流：我们先令所有负权边满流，然后按照上下界费用流的方式进行调整即可。显然在二次调整时不会出现负环，因此可以规约到不存在负环的费用流问题。

9.3. 二分图

二分图指可以被划分成两个集合 S, T 使得 $S \cup T = V, S \cap T = \emptyset$ ，且只有 S, T 之间存在边。二分图判定可以通过暴力染色在 $O(n + m)$ 的时间内完成。对于一个连通块二分图而言，对其进行二染色的方案数恰好为 2。

对于一张图 $G = (V, E)$ ，做出以下定义：

- 匹配：一张图的一组匹配是一个边集满足该边集中的边两两不共点。
- 边覆盖：一张图的一个边覆盖是一个边集满足该边集的端点的并是 V 。
- 独立集：一张图的一个独立集是一个点集满足任意一条边至多只有一个端点在点集中。
- 点覆盖：一张图的一个点覆盖是一个点集满足任意一条边至少有一个端点在点集中。

对于一般图而言，上述四个量很难求解，至少 NOI 不会考任意图的最大匹配 / 最小边覆盖 / 最大独立集 / 最小点覆盖。但是对于二分图而言，以上四个量就有比较好的性质可言了。显然，求解二分图的最大匹配有一个很简洁的建模方法：新建源汇，对于左部点 x ，连边 $S \rightarrow x$ ，容量为 1，对于右部点 y ，连边 $y \rightarrow T$ ，容量为 1。对于有边相连的左右部点 x, y ，连边 $x \rightarrow y$ ，容量为 1。然后跑最大流就是最大匹配。时间复杂度可以被证明是 $m\sqrt{n}$ 的，这里不再赘述，故二分图最大匹配最多可以处理 $n = 10^5, m = 10^5$ 的数据。

现在我们已经知道了二分图最大匹配怎么求，但是还有三个量我们是不知道的。考虑发掘它们之间的联系。可以证明二分图最大匹配与最小边覆盖的和等于 $|V|$ (前提是图不存在孤立点)，设最大匹配为 x ，那么我们考虑一个最大匹配 E' ，此时与这个 E' 关联的点数是 $2x$ ，那么显然我们再往 E' 中加入 $n - 2x$ 条边就可以使与这个 E' 关联的边数为 n ，这样我们总共选了 $n - 2x + x = n - x$ 条边。同理最大独立集与最小点覆盖的和也是 $|V|$ ，具体方法就是取最大独立集的补集就是最小点覆盖，这个很容易证明。这样我们已经得出了两组关系。事实上，我们还可以得到二分图最大匹配等于最小点覆盖，具体方法就是考虑残量网络上二分图左部能到达的点与右部不能到达的点，可以证明这是一个大小等于匹配数的点覆盖，这里不再赘述。这样对于二分图而言，我们知道上述一个量的值就可以知道并构造另外三个了。

当然还有一些其他与网络流与二分图有关的重要定理，限于篇幅原因这里不再证明：

- Hall 定理：二分图存在完美匹配的充要条件是对于任意一个左部点的集合 V 都有 $\text{neighbour}(V) \geq |V|$ ，其中 neighbour 为 V 的邻居集合。该定理在结论的证明中很有用。
- Dilworth 定理：对于任意一张满足 $x \rightarrow y$ 存在边， $y \rightarrow z$ 存在边可以推出 $x \rightarrow z$ 存在边的有向图，其最长反链（即任意两点都不可达）大小等于最小链覆盖，该定理可以对形如“最多选多少个 xxx 使得两两不可达”的问题转化为最小链覆盖问题。

9.4. 常见模型

个人认为，网络流最难的地方在于你需要发现它是个网络流，并为其设计相应的模型。就一般的方法而言，如果你发现一个问题数据范围很小，并且乍一看感觉是个 NPC 问题，可以考虑想想能不能网络流做。当然，了解一些很常用的网络流模型也会帮助你发现建模方法，这里稍微做个总结：

1. 集合划分模型：有 n 个元素，你要将它们分成两个集合 A, B ，有若干个事件形如“第 a_i 个元素如果分在第 b_i 个集合可以获得 c_i 的奖励”，还有若干个形如“如果 S_i 中的元素都被分在第 t_i 个集合则可以获得 w_i 的奖励”，最大化总奖励。考虑将原问题进行最小割转化，定义一个点与源点相连当且仅当其被划分到了 A ，与汇点相连当且仅当其被划分到了 B 。这样一来我们先假设所有奖励都可以拿得到，如果 $b_i = A$ 那么我们连边 $S \rightarrow a_i$ ，权值 c_i ，表示如果其与汇点相连就要少掉 c_i 的奖励，对于 $b_i = T$ 同理。对于与 S_i 有关的奖励我们新建一个虚点 X ，如果 $t_i = A$ ，那么我们连边 $S \rightarrow X$ 权值 w_i ，表示如果 X 与汇点相连那么会少掉 w_i 的奖励，以及 $\forall x \in S_i$ 连边 $X \rightarrow x$ ，容量 ∞ ，表示如果 S_i 中与 T 相连的点，那么 X 也与 T 相连，否则 X 与 S 相连。然后那 $\sum c_i + \sum w_i - \text{最小割}$ 即可。最大权闭合子图问题也是一类集合划分模型，其模型是每个点有一个权值 w_i ，有若干对关系 $a \rightarrow b$ 表示 a 选可以推出 b 选，求选一个子图的最小代价。将选看作划分入 S ，不选看作划分入集合 T ，这样对于 $w_i > 0$ 那么可以视作划分入 S 有 w_i 的奖励，对于 $w_i \leq 0$ 的先假设我们得到了 w_i 的负贡献，划分入集合 T 可以得到 $-w_i$ 的奖励，然后对于 $a \rightarrow b$ ，连边 $a \rightarrow b$ 容量 ∞ 然后拿 $\sum \max(w_i, 0) - \text{最小割}$ 就是答案。
2. 顶点上有容量限制：设第 i 个点的容量限制为 w_i ，将每个点拆成 in 和 out ，然后在 in 和 out 之间连 w_i 的边，然后对于原图中的边 (u, v) 连边 $out_u \rightarrow in_v$ 即可。

3. DAG 最小路径覆盖：对于不可交的路径覆盖，我们考虑这样的思想：先假设每个点单独一条路径，然后尽可能多地合并两条路径，考虑合并的办法，必然是找一个路径的终点和另一条路径的起点合并，并且每个点最多作为终点合并一次，每个点最多作为起点合并一次，因此考虑拆点二分图，我们将每个点 u 拆成两个 u 和 u' ，然后对于原图中的每条边 $u \rightarrow v$ ，连边 $u \rightarrow v'$ 。这样 n —二分图最大匹配 就是最小路径覆盖。具体构造就考虑每条匹配边 $u \rightarrow v'$ ，将以 u 为终点和以 v 为起点的路径合并起来即可。对于可相交的路径覆盖，一种做法是求出传递闭包后跑不可相交的路径覆盖，显然这和可相交的路径覆盖是等价的。另一种做法是拆点用上下界网络流处理，具体来说先拆点，对于所有 i 连边 $S \rightarrow in_i$ 容量 ∞ ， $out_i \rightarrow T$ 容量 ∞ ， $in_i \rightarrow out_i$ 下界 1 上界 ∞ ，对于原图中的边 u, v 连边 $out_u \rightarrow in_v$ 容量 ∞ ，然后跑最小流即可。
4. 导数建图法，对于一类最小费用问题，如果费用是关于流量的下凸函数 $c(f)$ ，那么可以考虑导数建图法，具体来说假设流量上界为 f ，那么我们连 f 条边，第 i 条边容量为 $c(i) - c(i-1)$ 。
5. 对于一些涉及到形如“选择一些区间，区间带权”，要求权值和的最大值的问题，并且要求每个点最多被 k 个区间覆盖，可以考虑建一个数轴一样的模型，然后将一个区间视作一条流，或者将一个区间覆盖的过程视作一条流，需根据实际情况进行选择，具体应用可见最长 k 可重区间集问题。
6. 对于一些数据范围比较大 ($10^5, 10^6$ 之类)，但是有一定性质的图，求解其最大流可以考虑模拟最大流。方法是模拟最小割，考虑割掉哪些边最优，如果我们能够发现其中的性质，并且用相应的数据结构维护，那么我们就可以实现快速求解求解特殊图的最大流。
7. 对于一些数据范围比较大的特殊图的费用流，可以考虑模拟费用流，具体方法就是反悔贪心，然后用若干个堆维护。